

Human review packet: GeEtAl2024AlignmentAxioms

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GeEtAl2024AlignmentAxioms

Paper id: GeEtAl2024AlignmentAxioms

Source version: NeurIPS 2024 published conference version

Source record: audit/paper_statement_map.json

Rows: 15

Generated: 2026-09-27

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GeEtAl2024AlignmentAxioms --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(90; supporting context\)](#)
- [Semantic library prerequisites \(22; not paper claims\)](#)
 - [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
 - [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
 - [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
 - [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
 - [AppliedModelingLib.Alignment.Axioms.SamePairwiseMajorityRelation](#)
 - [AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule](#)
 - [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
 - [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
 - [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
 - [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
 - [AppliedModelingLib.Alignment.Axioms.linearReward](#)
 - [AppliedModelingLib.Alignment.Axioms.InducesRanking](#)
 - [AppliedModelingLib.Alignment.Axioms.IsKemenySelector](#)
 - [AppliedModelingLib.Alignment.Axioms.NondegenerateParameter](#)
 - [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
 - [AppliedModelingLib.Alignment.Axioms.MajorityConsistent](#)
 - [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
 - [AppliedModelingLib.Alignment.Axioms.ParetoOptimal](#)
 - [AppliedModelingLib.Alignment.Axioms.RankingSeparability](#)
 - [AppliedModelingLib.Alignment.Axioms.WinnerMonotonic](#)
 - [AppliedModelingLib.Alignment.Axioms.canonicalKemenyRule](#)
 - [AppliedModelingLib.Alignment.Axioms.kemenyDisagreement](#)
- [Paper-specific semantic prerequisites \(9; not paper claims\)](#)
 - [GeEtAl2024AlignmentAxioms.IsLeximaxPluralitySelector](#)
 - [GeEtAl2024AlignmentAxioms.majorityLoss](#)

- GeEtAl2024AlignmentAxioms.globalMajorityLossInfimum
- GeEtAl2024AlignmentAxioms.restrictedMajorityLossInfimum
- GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing
- GeEtAl2024AlignmentAxioms.IsParetoKemenySelector
- GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralitySelector
- GeEtAl2024AlignmentAxioms.fixedTieLCPO
- GeEtAl2024AlignmentAxioms.standardLossFormulationSpec
- Main-text results (7)
 - theorem3_1_lossBasedImpossibilitySpec
 - lemma3_2_unconstrainedCoreSeparationSpec
 - lemma3_3_coreMinimizersSpec
 - lemma3_4_literalZeroWeakInclusionSpec
 - lemma3_5_correctedPerturbationGapSpec
 - theorem3_6_majorityLossSpec
 - theorem4_3_constructedFixedTieLCPOSpec
- Appendix results (6)
 - theoremC_2_copelandAndLCPOFailSeparabilitySpec
 - theoremC_3_linearKemenySpec
 - theoremC_4_linearKemenyFailsParetoAndMajorityConsistencySpec
 - theoremC_5_paretoKemenyFailsSeparabilityAndMajorityConsistencySpec
 - theoremC_6_fixedTieLeximaxPluralitySpec
 - appendixBExplicitInfeasiblePMCSpec
- Main-text results (continued) (2)
 - pmcRankingUniquenessAndNonexistenceSpec
 - theorem3_7_C1FailsParetoAndConsequencesSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Alignment.Axioms.votersSatisfying

Declaration location: reusable library

Lean declaration body

```
type:
  {Voter : Type u_1} → [Fintype Voter] → (predicate : Voter → Prop) → [DecidablePred predicate] → Finset Voter

value:
  fun {Voter : Type u_1} [Fintype Voter] (predicate : Voter → Prop) [DecidablePred predicate] =>
    Finset.filter predicate Finset.univ
```

AppliedModelingLib.Alignment.Axioms.StrictMajority

Declaration location: reusable library

Lean declaration body

```
type:
  {Voter : Type u_1} → [Fintype Voter] → (predicate : Voter → Prop) → [DecidablePred predicate] → Prop

value:
  fun {Voter : Type u_1} [Fintype Voter] (predicate : Voter → Prop) [DecidablePred predicate] =>
    Fintype.card Voter < 2 * (AppliedModelingLib.Alignment.Axioms.votersSatisfying predicate).card
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.votersSatisfying](#)

AppliedModelingLib.Alignment.Axioms.rankingProfileAppend

Declaration location: reusable library

Lean declaration body

```
type:
  {n leftCount rightCount : ℕ} →
    AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) n →
    AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) n →
    Fin (leftCount + rightCount) → AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
  fun {n leftCount rightCount : ℕ} (left : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) n)
    (right : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) n)
    (a : Fin (leftCount + rightCount)) =>
    Fin.addCases left right a
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.rankingTieKey

Declaration location: reusable library

Lean declaration body

```
type:
  {n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℕ

value:
  fun {n : ℕ} (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    ↑((Fintype.equivFin (AppliedModelingLib.SocialChoice.Ranking.Ranking n)) ranking)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.SocialChoice.Ranking.firstChoice

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n
value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) => π 0
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.MajorityTopChoice

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop
value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.Alignment.Axioms.StrictMajority fun (voter : Voter) =>
    AppliedModelingLib.SocialChoice.Ranking.firstChoice (profile voter) = candidate
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajority](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.firstChoice](#)

AppliedModelingLib.SocialChoice.Ranking.rankOf

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n
value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
(c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  (Equiv.symm π) c
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.ElevatesCandidate

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {n : ℕ} (original updated : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.SocialChoice.Ranking.rankOf updated candidate ≤
  AppliedModelingLib.SocialChoice.Ranking.rankOf original candidate ∧
  ∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  first ≠ candidate →
  second ≠ candidate →
  (AppliedModelingLib.Alignment.Axioms.StrictlyPrefers original first second ↔
  AppliedModelingLib.Alignment.Axioms.StrictlyPrefers updated first second)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [AppliedModelingLib.SocialChoice.Ranking.rankOf](#)

AppliedModelingLib.Alignment.Axioms.ProfileElevatesCandidate

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  Voter → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {Voter : Type u_1} {n : ℕ} (original updated : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (voter : Voter) (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.Alignment.Axioms.ElevatesCandidate (original voter) (updated voter) candidate ∧
  ∀ (otherVoter : Voter), otherVoter ≠ voter → updated otherVoter = original otherVoter
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.ElevatesCandidate](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.UniversallyPreferred

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {Voter : Type u_1} {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  ∀ (voter : Voter), AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (profile voter) first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) →
Decidable (AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ranking first second)

value:
fun {n : ℕ} (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers._aux_1 ranking first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [AppliedModelingLib.SocialChoice.Ranking.rankOf](#)

AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
AppliedModelingLib.Alignment.Axioms.StrictMajority fun (voter : Voter) =>
AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (profile voter) first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajority](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ranking first second ↔
AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers profile first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.instDecidableStrictMajorityPrefers

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[inst : Fintype Voter] →
{n : ℕ} →
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) →
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) →
Decidable (AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers profile first second)

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
id (id (id inferInstance))
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajority](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.votersSatisfying](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(output : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
feasible output ∧
∀ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
feasible contender →
AppliedModelingLib.Alignment.Axioms.kemenyDisagreement profile output ≤
AppliedModelingLib.Alignment.Axioms.kemenyDisagreement profile contender
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.kemenyDisagreement](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.IsContractionConsistentKemenySelector

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
((voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible) →
Prop

value:
fun {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule :
(voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible) =>
(∀ (voterCount : ℕ) (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin voterCount) n),
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible profile ((rule voterCount).run profile)) ∧
∀ {outerCount innerCount : ℕ} (outer : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin outerCount) n)
(inner : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin innerCount) n),
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible inner ((rule outerCount).run outer) →
(∀ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible inner contender →
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible outer contender) →
(rule innerCount).run inner = (rule outerCount).run outer
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.kemenyMinimizers

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
Finset (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>
Finset.filter (AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible profile) Finset.univ
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
[inst : Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
feasible fallback →
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) →
{ output : AppliedModelingLib.SocialChoice.Ranking.Ranking n //
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible profile output ∧
∀ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible profile contender →
AppliedModelingLib.Alignment.Axioms.rankingTieKey output ≤
AppliedModelingLib.Alignment.Axioms.rankingTieKey contender }

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (hfallback : feasible fallback)
```

```

(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>
let minimizers : Finset (AppliedModelingLib.SocialChoice.Ranking.Ranking n) :=
  AppliedModelingLib.Alignment.Axioms.kemenyMinimizers feasible profile;
have hminimizers : minimizers.Nonempty :=
  AppliedModelingLib.Alignment.Axioms.kemenyMinimizers_nonempty feasible fallback hfallback profile;
let tieKeys : Finset ℕ := Finset.image AppliedModelingLib.Alignment.Axioms.rankingTieKey minimizers;
have htieKeys : tieKeys.Nonempty :=
  AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection._proof_1 feasible profile hminimizers;
let key : ℕ := tieKeys.min' htieKeys;
have hkeyMem : key ∈ tieKeys := Finset.min' mem tieKeys htieKeys;
let output : AppliedModelingLib.SocialChoice.Ranking.Ranking n :=
  Classical.choose
    (AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection._proof_2 feasible profile htieKeys hkeyMem);
have houtput : output ∈ minimizers ∧ AppliedModelingLib.Alignment.Axioms.rankingTieKey output = key :=
  AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection._proof_3 feasible profile htieKeys hkeyMem;
(output,
  AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection._proof_4 feasible profile htieKeys hkeyMem houtput)

```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.kemenyMinimizers](#)
- [AppliedModelingLib.Alignment.Axioms.rankingTieKey](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAI2024AlignmentAxioms.AgreesBefore

Declaration location: paper-local

Lean declaration body

```

type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {n : ℕ} (reference contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (position : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  ∀ earlier < position, contender earlier = reference earlier

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAI2024AlignmentAxioms.CanFeasiblyPlaceAt

Declaration location: paper-local

Lean declaration body

```

type:
{n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (selectedPrefix : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  ∃ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  feasible contender ∧
  GeEtAI2024AlignmentAxioms.AgreesBefore selectedPrefix contender position ∧ contender position = candidate

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAI2024AlignmentAxioms.AgreesBefore](#)

GeEtAl2024AlignmentAxioms.IsGlobalMinimizer

Declaration location: paper-local

Lean declaration body

```
type:
{α : Type u_1} → (α → ℝ) → α → Prop

value:
fun {α : Type u_1} (objective : α → ℝ) (point : α) => ∀ (contender : α), objective point ≤ objective contender
```

GeEtAl2024AlignmentAxioms.RespectsPareto

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  ∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
    AppliedModelingLib.Alignment.Axioms.UniversallyPreferred profile first second →
    AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ranking first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.UniversallyPreferred](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  feasible ranking ∧ GeEtAl2024AlignmentAxioms.RespectsPareto profile ranking
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.RespectsPareto](#)

GeEtAl2024AlignmentAxioms.CanPlaceAt

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n →
AppliedModelingLib.SocialChoice.Ranking.Candidate n →
AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(selectedPrefix : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
(position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
∃ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking feasible profile contender ∧
GeEtAl2024AlignmentAxioms.AgreesBefore selectedPrefix contender position ∧ contender position = candidate
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.AgreesBefore](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)

GeEtAl2024AlignmentAxioms.appendixBCandidate

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
AppliedModelingLib.SocialChoice.Ranking.Candidate 5
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.appendixBMinus

Declaration location: paper-local

Lean declaration body

```
type:
Fin 3 → GeEtAl2024AlignmentAxioms.appendixBCandidate

value:
fun (index : Fin 3) => (↑index + 3, GeEtAl2024AlignmentAxioms.appendixBMinus._proof_2 index)
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBPMCRanking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 5

value:
((((Equiv.refl GeEtAl2024AlignmentAxioms.appendixBCandidate).trans (Equiv.swap 0 6)).trans (Equiv.swap 1 0)).trans
(Equiv.swap 2 1)).trans
(Equiv.swap 3 2)).trans
(Equiv.swap 4 5)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBPlus

Declaration location: paper-local

Lean declaration body

```
type:
Fin 3 → GeEtAl2024AlignmentAxioms.appendixBCandidate

value:
fun (index : Fin 3) => { ↑index, GeEtAl2024AlignmentAxioms.appendixBPlus._proof_2 index }
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBStar

Declaration location: paper-local

Lean declaration body

```
type:
GeEtAl2024AlignmentAxioms.appendixBCandidate

value:
(6, GeEtAl2024AlignmentAxioms.appendixBStar._proof_2)
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBFeatures

Declaration location: paper-local

Lean declaration body

```
type:
GeEtAl2024AlignmentAxioms.appendixBCandidate → Fin 3 → ℝ

value:
fun (a : GeEtAl2024AlignmentAxioms.appendixBCandidate) (a_1 : Fin 3) =>
  if a = GeEtAl2024AlignmentAxioms.appendixBPlus a_1 then 1
  else
    if a = GeEtAl2024AlignmentAxioms.appendixBMinus a_1 then -1
    else if a = GeEtAl2024AlignmentAxioms.appendixBStar then 1 / 4 else 0
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)
- [GeEtAl2024AlignmentAxioms.appendixBMinus](#)
- [GeEtAl2024AlignmentAxioms.appendixBPlus](#)
- [GeEtAl2024AlignmentAxioms.appendixBStar](#)

GeEtAl2024AlignmentAxioms.appendixBVoter1Ranking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 5

value:
((((Equiv.refl GeEtAl2024AlignmentAxioms.appendixBCandidate).trans (Equiv.swap 1 6)).trans (Equiv.swap 2 1)).trans
  (Equiv.swap 3 2)).trans
  (Equiv.swap 4 5)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBVoter2Ranking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 5

value:
((((((Equiv.refl GeEtAl2024AlignmentAxioms.appendixBCandidate).trans (Equiv.swap 0 1)).trans (Equiv.swap 0 6)).trans
  (Equiv.swap 2 0)).trans
  (Equiv.swap 3 2)).trans
  (Equiv.swap 4 5)).trans
  (Equiv.swap 4 3)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBVoter3Ranking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 5

value:
((((Equiv.refl GeEtAl2024AlignmentAxioms.appendixBCandidate).trans (Equiv.swap 0 2)).trans (Equiv.swap 1 6)).trans
  (Equiv.swap 3 1)).trans
  (Equiv.swap 5 3)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBCandidate](#)

GeEtAl2024AlignmentAxioms.appendixBProfile

Declaration location: paper-local

Lean declaration body

```
type:
Fin 3 → AppliedModelingLib.SocialChoice.Ranking.Ranking 5

value:
fun (a : Fin 3) =>
  match a with
  | (0, isLt) => GeEtAl2024AlignmentAxioms.appendixBVoter1Ranking
  | (1, isLt) => GeEtAl2024AlignmentAxioms.appendixBVoter2Ranking
  | (2, isLt) => GeEtAl2024AlignmentAxioms.appendixBVoter3Ranking
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBVoter1Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBVoter2Ranking](#)
- [GeEtAl2024AlignmentAxioms.appendixBVoter3Ranking](#)

GeEtAl2024AlignmentAxioms.c1Candidate

Declaration location: paper-local

Lean declaration body

type:
Type
value:
AppliedModelingLib.SocialChoice.Ranking.Candidate 7

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.c1BaseBallot

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 7
value:
(((Equiv.trans (Equiv.swap 1 8) (Equiv.swap 1 4)).trans (Equiv.swap 1 3)).trans (Equiv.swap 1 2)).trans (Equiv.swap 5 7)

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.c1Candidate](#)

GeEtAl2024AlignmentAxioms.c1Minus

Declaration location: paper-local

Lean declaration body

type:
Fin 4 → GeEtAl2024AlignmentAxioms.c1Candidate
value:
fun (index : Fin 4) => (↑index + 4, GeEtAl2024AlignmentAxioms.c1Minus._proof_2 index)

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.c1Candidate](#)

GeEtAl2024AlignmentAxioms.c1Order2134

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2
value:
Equiv.swap 0 1

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.c1Order3124

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2

value:
Equiv.trans (Equiv.swap 0 2) (Equiv.swap 0 1)

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.c1Order3241

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2

value:
Equiv.trans (Equiv.swap 0 2) (Equiv.swap 0 3)

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.c1Order4123

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2

value:
(Equiv.trans (Equiv.swap 0 3) (Equiv.swap 0 2)).trans (Equiv.swap 0 1)

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.c1Plus

Declaration location: paper-local

Lean declaration body

type:
Fin 4 → GeEtAl2024AlignmentAxioms.c1Candidate

value:
fun (index : Fin 4) => { ↑index, GeEtAl2024AlignmentAxioms.c1Plus._proof_2 index }

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.c1Candidate](#)

GeEtAl2024AlignmentAxioms.c1Star

Declaration location: paper-local

Lean declaration body

type:
GeEtAl2024AlignmentAxioms.c1Candidate

value:
(8, GeEtAl2024AlignmentAxioms.c1Star._proof_2)

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.c1Candidate](#)

GeEtAl2024AlignmentAxioms.c1CoordinateMap

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2 →
GeEtAl2024AlignmentAxioms.c1Candidate → GeEtAl2024AlignmentAxioms.c1Candidate

value:
fun (order : AppliedModelingLib.SocialChoice.Ranking.Ranking 2) (candidate : GeEtAl2024AlignmentAxioms.c1Candidate) =>
if hplus : ↑candidate < 4 then GeEtAl2024AlignmentAxioms.c1Plus (order (↑candidate, hplus))
else
if hminus : ↑candidate < 8 then
GeEtAl2024AlignmentAxioms.c1Minus
(order (↑candidate - 4, GeEtAl2024AlignmentAxioms.c1CoordinateMap._proof_2 candidate hplus hminus))
else GeEtAl2024AlignmentAxioms.c1Star

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.c1Candidate](#)
- [GeEtAl2024AlignmentAxioms.c1Minus](#)
- [GeEtAl2024AlignmentAxioms.c1Plus](#)
- [GeEtAl2024AlignmentAxioms.c1Star](#)

GeEtAl2024AlignmentAxioms.c1CoordinateRelabel

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2 → Equiv.Perm GeEtAl2024AlignmentAxioms.c1Candidate

value:
fun (order : AppliedModelingLib.SocialChoice.Ranking.Ranking 2) =>
{ toFun := GeEtAl2024AlignmentAxioms.c1CoordinateMap order,
invFun := GeEtAl2024AlignmentAxioms.c1CoordinateMap (Equiv.symm order),
left_inv := GeEtAl2024AlignmentAxioms.c1CoordinateRelabel._proof_1 order,
right_inv := GeEtAl2024AlignmentAxioms.c1CoordinateRelabel._proof_2 order }

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.c1Candidate](#)
- [GeEtAl2024AlignmentAxioms.c1CoordinateMap](#)

GeEtAl2024AlignmentAxioms.c1Ballot

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 2 → AppliedModelingLib.SocialChoice.Ranking.Ranking 7
value:
fun (order : AppliedModelingLib.SocialChoice.Ranking.Ranking 2) =>
Equiv.trans GeEtAl2024AlignmentAxioms.c1BaseBallot (GeEtAl2024AlignmentAxioms.c1CoordinateRelabel order)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.c1BaseBallot](#)
- [GeEtAl2024AlignmentAxioms.c1CoordinateRelabel](#)

GeEtAl2024AlignmentAxioms.c1Features

Declaration location: paper-local

Lean declaration body

```
type:
GeEtAl2024AlignmentAxioms.c1Candidate → Fin 4 → ℝ
value:
fun (a : GeEtAl2024AlignmentAxioms.c1Candidate) (a_1 : Fin 4) =>
if a = GeEtAl2024AlignmentAxioms.c1Plus a_1 then 1
else
if a = GeEtAl2024AlignmentAxioms.c1Minus a_1 then -1 else if a = GeEtAl2024AlignmentAxioms.c1Star then 1 / 5 else 0
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.c1Candidate](#)
- [GeEtAl2024AlignmentAxioms.c1Minus](#)
- [GeEtAl2024AlignmentAxioms.c1Plus](#)
- [GeEtAl2024AlignmentAxioms.c1Star](#)

GeEtAl2024AlignmentAxioms.c1ProfilePlus1

Declaration location: paper-local

Lean declaration body

```
type:
Fin 5 → AppliedModelingLib.SocialChoice.Ranking.Ranking 7
value:
fun (a : Fin 5) =>
match a with
| (0, isLt) => GeEtAl2024AlignmentAxioms.c1Ballot GeEtAl2024AlignmentAxioms.c1Order2134
| (1, isLt) => GeEtAl2024AlignmentAxioms.c1Ballot GeEtAl2024AlignmentAxioms.c1Order3124
| (2, isLt) => GeEtAl2024AlignmentAxioms.c1Ballot GeEtAl2024AlignmentAxioms.c1Order3241
| (3, isLt) => GeEtAl2024AlignmentAxioms.c1Ballot GeEtAl2024AlignmentAxioms.c1Order4123
| (4, isLt) => GeEtAl2024AlignmentAxioms.c1Ballot GeEtAl2024AlignmentAxioms.c1Order4123
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.c1Ballot](#)
- [GeEtAl2024AlignmentAxioms.c1Order2134](#)
- [GeEtAl2024AlignmentAxioms.c1Order3124](#)
- [GeEtAl2024AlignmentAxioms.c1Order3241](#)
- [GeEtAl2024AlignmentAxioms.c1Order4123](#)

GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 5 → Prop
value:
fun (a : AppliedModelingLib.SocialChoice.Ranking.Ranking 5) => True
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.c2Candidate

Declaration location: paper-local

Lean declaration body

```
type:
Type
value:
AppliedModelingLib.SocialChoice.Ranking.Candidate 5
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.c4Candidate

Declaration location: paper-local

Lean declaration body

```
type:
Type
value:
AppliedModelingLib.SocialChoice.Ranking.Candidate 18
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.c4Features

Declaration location: paper-local

Lean declaration body

```
type:
GeEtAl2024AlignmentAxioms.c4Candidate → Fin 7 → ℝ
value:
fun (a : GeEtAl2024AlignmentAxioms.c4Candidate) (a_1 : Fin 7) =>
  ![![2000000, 0, 0, 0, 0, 0, 0], ![0, 2000000, 0, 0, 0, 0, 0], ![0, 200000, 0, 0, 0, 0, 0],
    ![0, 100000, 100000, 0, 0, 0, 0], ![0, 0, 200000, 0, 0, 0, 0], ![0, 0, 20000, 0, 0, 0, 0],
    ![0, 0, 10000, 10000, 0, 0, 0], ![0, 0, 0, 20000, 0, 0, 0], ![0, 0, 0, 2000, 0, 0, 0],
    ![0, 0, 0, 1000, 1000, 0, 0], ![0, 0, 0, 0, 2000, 0, 0], ![0, 0, 0, 0, 200, 0, 0], ![0, 0, 0, 0, 100, 100, 0],
    ![0, 0, 0, 0, 0, 200, 0], ![0, 0, 0, 0, 0, 20, 0], ![0, 0, 0, 0, 0, 10, 10], ![0, 0, 0, 0, 0, 20],
    ![0, 0, 0, 0, 0, 2], ![1, 0, 0, 0, 0, 0, 1], ![2, 0, 0, 0, 0, 0, 0]]
a a_1
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [GeEtAl2024AlignmentAxioms.c4Candidate](#)

GeEtAl2024AlignmentAxioms.IsParetoKemenyMinimizer

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter 18 →
  AppliedModelingLib.SocialChoice.Ranking.Ranking 18 → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter 18)
  (output : AppliedModelingLib.SocialChoice.Ranking.Ranking 18) =>
  GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) profile output ∧
  ∀ (contender : AppliedModelingLib.SocialChoice.Ranking.Ranking 18),
    GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) profile
      contender →
    AppliedModelingLib.Alignment.Axioms.kemenyDisagreement profile output ≤
    AppliedModelingLib.Alignment.Axioms.kemenyDisagreement profile contender
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.kemenyDisagreement](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)
- [GeEtAl2024AlignmentAxioms.c4Features](#)

GeEtAl2024AlignmentAxioms.copelandScore

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
  {n : ℕ} →
    AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  {opponent : AppliedModelingLib.SocialChoice.Ranking.Candidate n |
    AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers profile candidate opponent}.card
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictMajorityPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.CopelandBetterWith

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Candidate n →
AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
GeEtAl2024AlignmentAxioms.copelandScore profile second < GeEtAl2024AlignmentAxioms.copelandScore profile first ∨
GeEtAl2024AlignmentAxioms.copelandScore profile first = GeEtAl2024AlignmentAxioms.copelandScore profile second ∧
priority first < priority second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.copelandScore](#)

GeEtAl2024AlignmentAxioms.IsLCPOOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(outcome : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking feasible profile outcome ∧
∀ (position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
GeEtAl2024AlignmentAxioms.CanPlaceAt feasible profile outcome position candidate →
GeEtAl2024AlignmentAxioms.copelandScore profile candidate ≤
GeEtAl2024AlignmentAxioms.copelandScore profile (outcome position)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.CanPlaceAt](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)
- [GeEtAl2024AlignmentAxioms.copelandScore](#)

GeEtAl2024AlignmentAxioms.IsLCPOSelector

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
GeEtAl2024AlignmentAxioms.IsLCPOOutcome feasible profile (rule.run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsLCPOOutcome](#)

GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(outcome : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking feasible profile outcome ∧
∀ (position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
GeEtAl2024AlignmentAxioms.CanPlaceAt feasible profile outcome position candidate →
¬GeEtAl2024AlignmentAxioms.CopelandBetterWith priority profile candidate (outcome position)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.CanPlaceAt](#)
- [GeEtAl2024AlignmentAxioms.CopelandBetterWith](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)

GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOSelector

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) =>
Function.Injective priority ∧
∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOOutcome feasible priority profile (rule.run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOOutcome](#)

GeEtAI2024AlignmentAxioms.SameCopelandScoreOrder

Declaration location: paper-local

Lean declaration body

```
type:
{leftCount rightCount : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) 5 →
  AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) 5 → Prop

value:
fun {leftCount rightCount : ℕ} (left : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) 5)
  (right : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) 5) =>
  ∀ (first second : GeEtAI2024AlignmentAxioms.c2Candidate),
    GeEtAI2024AlignmentAxioms.copelandScore left first < GeEtAI2024AlignmentAxioms.copelandScore left second ↔
    GeEtAI2024AlignmentAxioms.copelandScore right first < GeEtAI2024AlignmentAxioms.copelandScore right second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [GeEtAI2024AlignmentAxioms.c2Candidate](#)
- [GeEtAI2024AlignmentAxioms.copelandScore](#)

GeEtAI2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector

Declaration location: paper-local

Lean declaration body

```
type:
((voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
  GeEtAI2024AlignmentAxioms.c2AllRankingsFeasible) →
  Prop

value:
fun
  (rule :
    (voterCount : ℕ) →
      AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
      GeEtAI2024AlignmentAxioms.c2AllRankingsFeasible) =>
  (∀ (voterCount : ℕ) (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin voterCount) 5)
    (first second : GeEtAI2024AlignmentAxioms.c2Candidate),
      GeEtAI2024AlignmentAxioms.copelandScore profile second < GeEtAI2024AlignmentAxioms.copelandScore profile first →
      AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ((rule voterCount).run profile) first second) ∧
  ∀ (leftCount rightCount : ℕ) (left : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) 5)
    (right : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) 5),
    GeEtAI2024AlignmentAxioms.SameCopelandScoreOrder left right →
    (rule leftCount).run left = (rule rightCount).run right
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAI2024AlignmentAxioms.SameCopelandScoreOrder](#)
- [GeEtAI2024AlignmentAxioms.c2AllRankingsFeasible](#)
- [GeEtAI2024AlignmentAxioms.c2Candidate](#)
- [GeEtAI2024AlignmentAxioms.copelandScore](#)

GeEtAI2024AlignmentAxioms.IsConsistentlyTieBrokenLCPOSelector

Declaration location: paper-local

Lean declaration body

```
type:
((voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
  GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible) →
((voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
  GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible) →
Prop

value:
fun
  (copelandRule lcpoRule :
    (voterCount : ℕ) →
      AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
      GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible) =>
  GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector copelandRule ∧
  (∀ (voterCount : ℕ),
    GeEtAl2024AlignmentAxioms.IsLCPOSelector GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible
    (lcpoRule voterCount)) ∧
  ∀ (voterCount : ℕ) (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin voterCount) 5),
    GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible profile
    ((copelandRule voterCount).run profile) →
    (lcpoRule voterCount).run profile = (copelandRule voterCount).run profile
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector](#)
- [GeEtAl2024AlignmentAxioms.IsLCPOSelector](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)
- [GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible](#)

GeEtAl2024AlignmentAxioms.copelandTieKey

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  GeEtAl2024AlignmentAxioms.copelandScore profile candidate *
  Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n) +
  (Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n) - 1 - ↑candidate)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.copelandScore](#)

GeEtAl2024AlignmentAxioms.copelandTieKeyVector

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ

value:
```

```

fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
toLex
  (fun (position : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    GeEtAl2024AlignmentAxioms.copelandTieKey profile (ranking position))
a

```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.copelandTieKey](#)

GeEtAl2024AlignmentAxioms.copyBelowOriginalCone

Declaration location: paper-local

Lean declaration body

```

type:
ℝ → ℝ × ℝ → Prop

value:
fun (δ : ℝ) (parameter : ℝ × ℝ) => -parameter.1 + δ * parameter.2 < 0

```

GeEtAl2024AlignmentAxioms.coreFeatureRewards

Declaration location: paper-local

Lean declaration body

```

type:
ℝ × ℝ → ℝ × ℝ

value:
fun (parameter : ℝ × ℝ) => (2 * parameter.1 + parameter.2, parameter.1 + parameter.2)

```

GeEtAl2024AlignmentAxioms.inducingParameters

Declaration location: paper-local

Lean declaration body

```

type:
{n dimension : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → Prop

value:
fun {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (a : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
  {parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension |
    AppliedModelingLib.Alignment.Axioms.InducesRanking features parameter ranking}
a

```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.InducesRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAl2024AlignmentAxioms.paretoFeasibleRankings

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
Finset (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>
Finset.filter (GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking feasible profile) Finset.univ
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.ParetoFeasibleRanking](#)

GeEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
[Nonempty Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) →
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ

value:
fun {Voter : Type u_1} [Fintype Voter] [Nonempty Voter] {n : ℕ}
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(hprofile : AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile)
(a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
(let searchDomain : Finset (AppliedModelingLib.SocialChoice.Ranking.Ranking n) :=
GeEtAl2024AlignmentAxioms.paretoFeasibleRankings feasible profile;
let keyVectors : Finset (Lex (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)) :=
Finset.image (GeEtAl2024AlignmentAxioms.copelandTieKeyVector profile) searchDomain;
have hsearchDomain : searchDomain.Nonempty :=
GeEtAl2024AlignmentAxioms.paretoFeasibleRankings_nonempty feasible profile hprofile;
have hkeyVectors : keyVectors.Nonempty :=
GeEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector._proof_1 feasible profile hsearchDomain;
keyVectors.max' hkeyVectors)
a
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.copelandTieKeyVector](#)
- [GeEtAl2024AlignmentAxioms.paretoFeasibleRankings](#)

GeEtAl2024AlignmentAxioms.fixedTieLCPOSelection

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[inst : Fintype Voter] →
[inst_1 : Nonempty Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) →
(hprofile : AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile) →
{ ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n //
  ranking ∈ GetEtAl2024AlignmentAxioms.paretoFeasibleRankings feasible profile ∧
  GetEtAl2024AlignmentAxioms.copelandTieKeyVector profile ranking =
  GetEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector feasible profile hprofile }

value:
fun {Voter : Type u_1} [Fintype Voter] [Nonempty Voter] {n : ℕ}
  (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (hprofile : AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile) =>
let searchDomain : Finset (AppliedModelingLib.SocialChoice.Ranking.Ranking n) :=
  GetEtAl2024AlignmentAxioms.paretoFeasibleRankings feasible profile;
let keyVectors : Finset (Lex (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)) :=
  Finset.image (GetEtAl2024AlignmentAxioms.copelandTieKeyVector profile) searchDomain;
have hsearchDomain : searchDomain.Nonempty :=
  GetEtAl2024AlignmentAxioms.paretoFeasibleRankings_nonempty feasible profile hprofile;
have hkeyVectors : keyVectors.Nonempty :=
  GetEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector._proof_1 feasible profile hsearchDomain;
have hExists :
  ∃ ranking ∈ searchDomain,
    GetEtAl2024AlignmentAxioms.copelandTieKeyVector profile ranking =
    GetEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector feasible profile hprofile :=
  GetEtAl2024AlignmentAxioms.fixedTieLCPOSelection._proof_1 feasible profile hprofile hkeyVectors;
(Classical.choose hExists, GetEtAl2024AlignmentAxioms.fixedTieLCPOSelection._proof_2 feasible profile hprofile hExists)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GetEtAl2024AlignmentAxioms.copelandTieKeyVector](#)
- [GetEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector](#)
- [GetEtAl2024AlignmentAxioms.paretoFeasibleRankings](#)

GetEtAl2024AlignmentAxioms.fixedTieLCPOOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
[Nonempty Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) →
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
fun {Voter : Type u_1} [Fintype Voter] [Nonempty Voter] {n : ℕ}
  (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (hprofile : AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile) =>
  ↑(GetEtAl2024AlignmentAxioms.fixedTieLCPOSelection feasible profile hprofile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GetEtAl2024AlignmentAxioms.copelandTieKeyVector](#)
- [GetEtAl2024AlignmentAxioms.fixedTieLCPOKeyVector](#)
- [GetEtAl2024AlignmentAxioms.fixedTieLCPOSelection](#)
- [GetEtAl2024AlignmentAxioms.paretoFeasibleRankings](#)

GeEtAI2024AlignmentAxioms.pluralityScore

Declaration location: paper-local

Lean declaration body

```
type:
  {Voter : Type u_1} →
  [Fintype Voter] →
  {n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ

value:
  fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
    (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    ∑ voter : Voter, if AppliedModelingLib.SocialChoice.Ranking.firstChoice (profile voter) = candidate then 1 else 0
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.firstChoice](#)

GeEtAI2024AlignmentAxioms.IsLeximaxPluralityOutcome

Declaration location: paper-local

Lean declaration body

```
type:
  {Voter : Type u_1} →
  [Fintype Voter] →
  {n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
  fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
    (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
    (outcome : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    feasible outcome ∧
    ∀ (position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
    GeEtAI2024AlignmentAxioms.CanFeasiblyPlaceAt feasible outcome position candidate →
    GeEtAI2024AlignmentAxioms.pluralityScore profile candidate ≤
    GeEtAI2024AlignmentAxioms.pluralityScore profile (outcome position)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAI2024AlignmentAxioms.CanFeasiblyPlaceAt](#)
- [GeEtAI2024AlignmentAxioms.pluralityScore](#)

GeEtAI2024AlignmentAxioms.PluralityBetterWith

Declaration location: paper-local

Lean declaration body

```
type:
  {Voter : Type u_1} →
  [Fintype Voter] →
  {n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
  fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)
    (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
    (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    GeEtAI2024AlignmentAxioms.pluralityScore profile second < GeEtAI2024AlignmentAxioms.pluralityScore profile first ∨
    GeEtAI2024AlignmentAxioms.pluralityScore profile first = GeEtAI2024AlignmentAxioms.pluralityScore profile second ∧
    priority first < priority second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.pluralityScore](#)

GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralityOutcome

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(outcome : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
feasible outcome ∧
∀ (position candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
GeEtAl2024AlignmentAxioms.CanFeasiblyPlaceAt feasible outcome position candidate →
¬GeEtAl2024AlignmentAxioms.PluralityBetterWith priority profile candidate (outcome position)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.CanFeasiblyPlaceAt](#)
- [GeEtAl2024AlignmentAxioms.PluralityBetterWith](#)

GeEtAl2024AlignmentAxioms.positiveInputFeatures

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Candidate 0 → Fin 2 → ℝ

value:
fun (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) (a_1 : Fin 2) =>
if a = 0 then if a_1 = 0 then 1 else 0 else if a_1 = 0 then 0 else 1
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.rankingPairLoss

Declaration location: paper-local

Lean declaration body

```
type:
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.SocialChoice.Ranking.Ranking n →
(ℝ → ℝ) → AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → ℝ

value:
fun {n dimension : ℕ}
(features :
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (loss : ℝ → ℝ)
(parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
Σ pair : AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n,
if AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ranking pair.1 pair.2 then
loss
(AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.2 -
AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.1)
else 0
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAI2024AlignmentAxioms.sixCandidateCopyFeatures

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → AppliedModelingLib.SocialChoice.Ranking.Candidate 4 → Fin 2 → ℝ

value:
fun (ε 6 : ℝ) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 4) (a_1 : Fin 2) =>
  if a = 0 then if a_1 = 0 then 2 else 1
  else
    if a = 1 then if a_1 = 0 then 2 - ε else 1
    else
      if a = 2 then if a_1 = 0 then 1 else 1
      else if a = 3 then if a_1 = 0 then 1 - ε else 1 else if a = 4 then if a_1 = 0 then -ε else 6 * ε else 0
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAI2024AlignmentAxioms.sixCandidateForwardRanking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 4

value:
Equiv.refl (AppliedModelingLib.SocialChoice.Ranking.Candidate 4)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAI2024AlignmentAxioms.sixCandidateReverseRanking

Declaration location: paper-local

Lean declaration body

```
type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 4

value:
Equiv.trans (Equiv.swap 0 1) Fin.revPerm
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

GeEtAI2024AlignmentAxioms.sixCandidateWeightedLoss

Declaration location: paper-local

Lean declaration body

```
type:
{dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate 4 → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
ℝ → (ℝ → ℝ) → AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → ℝ

value:
fun {dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate 4 → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (weight : ℝ) (loss : ℝ → ℝ) (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
  weight *
    GeEtAl2024AlignmentAxioms.rankingPairLoss features GeEtAl2024AlignmentAxioms.sixCandidateForwardRanking loss
  parameter +
  (1 - weight) *
    GeEtAl2024AlignmentAxioms.rankingPairLoss features GeEtAl2024AlignmentAxioms.sixCandidateReverseRanking loss
  parameter
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.rankingPairLoss](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateForwardRanking](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateReverseRanking](#)

GeEtAl2024AlignmentAxioms.standardLoss

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
  AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  (ℝ → ℝ) → AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ)
  (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
  ∑ voter : Voter,
  ∑ pair : AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n,
  if AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (profile voter) pair.1 pair.2 then
    loss
      (AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.2 -
        AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.1)
  else 0
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

GeEtAl2024AlignmentAxioms.globalStandardLossInfimum

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → (ℝ → ℝ) → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
(features :
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ) =>
sInf (Set.range (GeEtAl2024AlignmentAxioms.standardLoss features profile loss))
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.standardLoss](#)

GeEtAl2024AlignmentAxioms.restrictedStandardLossInfimum

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
(ℝ → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
(features :
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ)
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
sInf
(GeEtAl2024AlignmentAxioms.standardLoss features profile loss "
{parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension |
AppliedModelingLib.Alignment.Axioms.InducesRanking features parameter ranking})
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.InducesRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.standardLoss](#)

GeEtAl2024AlignmentAxioms.IsStandardLossMinimizing

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(features :
  AppliedModelingLib.SocialChoice.Ranking.Candidate n →
  AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
( $\mathbb{R} \rightarrow \mathbb{R}$ ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) →
  Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (loss :  $\mathbb{R} \rightarrow \mathbb{R}$ )
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features)) =>
  ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
  AppliedModelingLib.Alignment.Axioms.FeasibleProfile
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) profile →
  GeEtAl2024AlignmentAxioms.restrictedStandardLossInfimum features profile loss (rule.run profile) =
  GeEtAl2024AlignmentAxioms.globalStandardLossInfimum features profile loss
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.globalStandardLossInfimum](#)
- [GeEtAl2024AlignmentAxioms.restrictedStandardLossInfimum](#)

GeEtAl2024AlignmentAxioms.twoCoordinateParameter

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \times \mathbb{R} \rightarrow \text{Fin } 2 \rightarrow \mathbb{R}$ 

value:
fun (parameter :  $\mathbb{R} \times \mathbb{R}$ ) (a : Fin 2) => if a = 0 then parameter.1 else parameter.2
```

GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (ε δ weight : ℝ) (loss :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (parameter :  $\mathbb{R} \times \mathbb{R}$ ) =>
  GeEtAl2024AlignmentAxioms.sixCandidateWeightedLoss (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures ε δ) weight
  loss (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateWeightedLoss](#)
- [GeEtAl2024AlignmentAxioms.twoCoordinateParameter](#)

GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss

Declaration location: paper-local

Lean declaration body

```
type:
( $\mathbb{R} \rightarrow \mathbb{R}$ )  $\rightarrow \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (rewards :  $\mathbb{R} \times \mathbb{R}$ ) => g (rewards.1 - rewards.2) + g rewards.1 + g rewards.2
```

GeEtAl2024AlignmentAxioms.coreLoss

Declaration location: paper-local

Lean declaration body

```
type:
( $\mathbb{R} \rightarrow \mathbb{R}$ )  $\rightarrow \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (parameter :  $\mathbb{R} \times \mathbb{R}$ ) =>
  GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss g (GeEtAl2024AlignmentAxioms.coreFeatureRewards parameter)
```

Direct retained dependencies

- [GeEtAl2024AlignmentAxioms.coreFeatureRewards](#)
- [GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss](#)

GeEtAl2024AlignmentAxioms.weightedLoss

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (p :  $\mathbb{R}$ ) (loss :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (x :  $\mathbb{R}$ ) => p * loss (-x) + (1 - p) * loss x
```

Material library prerequisites

AppliedModelingLib.SocialChoice.Ranking.Candidate

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model
Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.
Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.
For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]
collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.
We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .
3 Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.
We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that θ^* satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how θ^* ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.
4 We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

```
type:
ℕ → Type

value:
fun (n : ℕ) => Fin (n + 2)
```

Exact Lean library declaration

```
/--
Candidate set with at least two candidates. The parameter `n` means there are
`n + 2` candidates.
-/
abbrev Candidate (n : ℕ) := Fin (n + 2)
```

Recorded source-to-library screening Verdict: matches

Reason: The source uses a finite set C of distinct candidates; $\text{Fin } (n + 2)$ is a concrete finite indexing of such a candidate set and adds no preference or reward claim.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.SocialChoice.Ranking.Ranking

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \triangleright_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b: a \triangleright_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \triangleright_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:
 $\mathbb{N} \rightarrow \text{Type}$

value:
fun (n : $\mathbb{N}$) => Equiv.Perm (AppliedModelingLib.SocialChoice.Ranking.Candidate n)

Exact Lean library declaration

-- A ranking is a permutation of the candidate set. -/
abbrev Ranking (n : $\mathbb{N}$) := Equiv.Perm (Candidate n)

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-library screening Verdict: matches

Reason: The source treats rankings as permutations of the m candidates; Equiv.Perm Candidate is exactly a permutation ranking.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.RankingProfile

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \triangleright_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b: a \triangleright_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \triangleright_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:
Type u_1 → ℕ → Type u_1

value:
fun (Voter : Type u_1) (n : ℕ) => Voter → AppliedModelingLib.SocialChoice.Ranking.Ranking n

Exact Lean library declaration

-- A profile assigns one complete ranking to each voter. -/
abbrev RankingProfile (Voter : Type*) (n : ℕ) := Voter → Ranking n

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: The source defines the profile as the voter-indexed vector $\pi_i = (\sigma_i)$; Voter to Ranking is that representation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean declaration type (metadata)

field run:

```
{Voter : Type u_1} →
{n : ℕ} →
{feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop} →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → AppliedModelingLib.SocialChoice.Ranking.Ranking n
```

field output_feasible:

```
∀ {Voter : Type u_1} {n : ℕ} {feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop}
(self : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n), feasible (self.run profile)
```

Exact Lean library declaration

/--

A linear-rank-aggregation rule with an explicit output-feasibility guarantee.
The feature vectors/parameters which justify `feasible` stay external to this generic social-choice interface.

-/

```
structure LinearRankAggregationRule (Voter : Type*) (n : ℕ)
  (feasible : Ranking n → Prop) where
  run : RankingProfile Voter n → Ranking n
  output_feasible : ∀ profile, feasible (run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: The source's linear rank aggregation rule maps a profile to a feasible ranking; run plus output_feasible gives precisely that total-rule requirement.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.SamePairwiseMajorityRelation

Source locator: cited publication, lines 196-198

Verbatim paper-source connection

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b.

[Next verbatim source excerpt]

However, despite this good news for PMC, we show that this does not help in achieving PO. In fact, our negative result extends to every C1 linear rank aggregation rule. Note that if f minimizing ℓ in the majority formulation breaks ties consistently (i.e., if multiple feasible rankings are optimal, then it consistently chooses the same one), then it is C1. We then have the following result, whose proof is relegated to Appendix A.6.

Lean-expanded library semantic target

```
type:
  {Voter : Type u_1} →
  [Fintype Voter] →
  {n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → Prop
value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ}
  (firstProfile secondProfile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>
  ∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers firstProfile first second ↔
  AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers secondProfile first second
```

Exact Lean library declaration

```
-- Two profiles induce the same directed strict-pairwise-majority relation. -/
def SamePairwiseMajorityRelation {Voter : Type*} [Fintype Voter] {n : ℕ}
  (firstProfile secondProfile : RankingProfile Voter n) : Prop :=
  ∀ first second,
  StrictMajorityPrefers firstProfile first second ↔
  StrictMajorityPrefers secondProfile first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared the C1 source description and its Theorem 3.7 uses. The expanded relation checks, for every ordered candidate pair, equivalence of the strict-more-than-half voter condition in the two profiles. It therefore identifies exactly the same directed majority graph, including majority ties, and no cardinal or ranking information beyond that graph.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule

Source locator: cited publication, lines 196-198

Verbatim paper-source connection

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b.

[Next verbatim source excerpt]

However, despite this good news for PMC, we show that this does not help in achieving PO. In fact, our negative result extends to every C1 linear rank aggregation rule. Note that if f minimizing ℓ in the majority formulation breaks ties consistently (i.e., if multiple feasible rankings are optimal, then it consistently chooses the same one), then it is C1. We then have the following result, whose proof is relegated to Appendix A.6.

Lean-expanded library semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
  ∀ (firstProfile secondProfile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
    AppliedModelingLib.Alignment.Axioms.SamePairwiseMajorityRelation firstProfile secondProfile →
    rule.run firstProfile = rule.run secondProfile
```

Exact Lean library declaration

```
/--
The paper's `C1` condition: a rule's output depends only on the directed
strict-pairwise-majority relation of its input profile.
-/
def C1LinearRankAggregationRule {Voter : Type*} [Fintype Voter] {n : ℕ}
  (feasible : Ranking n → Prop) (rule : LinearRankAggregationRule Voter n feasible) : Prop :=
  ∀ firstProfile secondProfile,
    SamePairwiseMajorityRelation firstProfile secondProfile →
    rule.run firstProfile = rule.run secondProfile
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.SamePairwiseMajorityRelation](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared the source definition that a C1 rule depends only on pairwise majority relationships and all Theorem 3.7/consequence uses. The target compares every directed strict-majority edge in two profiles and requires identical rule outputs exactly when those relations coincide; tied pairs are represented by absence of either strict edge, so no majority information is omitted or added.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.FeasibleProfile

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) > r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:

```
{Voter : Type u_1} →  
{n : ℕ} →  
  (AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →  
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → Prop
```

value:

```
fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)  
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>  
  ∀ (voter : Voter), feasible (profile voter)
```

Exact Lean library declaration

/-- A profile whose submitted rankings all belong to a supplied feasible set. -/

```
def FeasibleProfile {Voter : Type*} {n : ℕ}  
  (feasible : Ranking n → Prop) (profile : RankingProfile Voter n) : Prop :=  
  ∀ voter, feasible (profile voter)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: The model says each submitted voter ranking is feasible; this is exactly the universal assertion that every profile entry satisfies the supplied feasibility predicate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by $\mathbb{R}^d$ the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in \mathbb{R}^d$. A parameter vector $\theta \in \mathbb{R}^d$ induces a linear reward function $r_\theta : C \rightarrow \mathbb{R}$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split $\mathbb{R}^d$ into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of $\mathbb{R}^d$, covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:
 $\mathbb{N} \rightarrow \text{Type}$

value:
fun (dimension : $\mathbb{N}$) => Fin dimension $\rightarrow \mathbb{R}$

Exact Lean library declaration

-- A finite-dimensional real feature vector. -/
abbrev FeatureVector (dimension : $\mathbb{N}$) := Fin dimension $\rightarrow \mathbb{R}$

Recorded source-to-library screening Verdict: matches

Reason: The source places candidate features in $\mathbb{R}^d$; Fin dimension to $\mathbb{R}$ is its finite-coordinate representation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.LinearRewardParameter

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R^d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R^d$. A parameter vector $\theta \in R^d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R^d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \triangleright_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b: a \triangleright_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R^d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \triangleright_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:
 $\mathbb{N} \rightarrow \text{Type}$

value:
fun (dimension : $\mathbb{N}$) => AppliedModelingLib.Alignment.Axioms.FeatureVector dimension

Exact Lean library declaration

-- A parameter vector for a linear reward model. -/
abbrev LinearRewardParameter (dimension : $\mathbb{N}$) := FeatureVector dimension

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)

Recorded source-to-library screening Verdict: matches

Reason: The source takes a parameter θ in R^d ; this abbreviation is the same finite-dimensional real coordinate space.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.StrictlyPrefers

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model
Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.
Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.
For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]
collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.
We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .
Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $na>b(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $wa>b(\pi) = na>b(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $na>b$ and $wa>b$, respectively.
We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.
We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {n : ℕ} (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.SocialChoice.Ranking.rankOf ranking first <
  AppliedModelingLib.SocialChoice.Ranking.rankOf ranking second
```

Exact Lean library declaration

```
/-- Candidate `first` is strictly above `second` in a ranking. -/
def StrictlyPrefers {n : ℕ} (ranking : Ranking n)
  (first second : Candidate n) : Prop :=
  rankOf ranking first < rankOf ranking second
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [AppliedModelingLib.SocialChoice.Ranking.rankOf](#)

Recorded source-to-library screening Verdict: matches

Reason: The source relation a succeeds σ if b is strict rank order; Lean compares their inverse-permutation positions, so smaller position means the same strict preference.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.linearReward

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow \mathbb{R}$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b: a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) > r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) := n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

type:

```
{n dimension : ℕ} →
  AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n →
      AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
      AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
```

value:

```
fun {n dimension : ℕ} (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension)
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  ∑ coordinate : Fin dimension, parameter coordinate * features candidate coordinate
```

Exact Lean library declaration

-- The dot-product reward of one candidate under a feature-linear model. -/

```
noncomputable def linearReward {n dimension : ℕ}
  (parameter : LinearRewardParameter dimension)
  (features : Candidate n → FeatureVector dimension)
  (candidate : Candidate n) : ℝ :=
  ∑ coordinate, parameter coordinate * features candidate coordinate
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-library screening Verdict: matches

Reason: The source defines $r_\theta(c)$ as the dot product of θ and the candidate feature vector; the finite coordinate sum is exactly that dot product.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

AppliedModelingLib.Alignment.Axioms.InducesRanking

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie. Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = (\theta, x_c)$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other. For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively. We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

```
type:
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
  AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension)
  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  ∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
    AppliedModelingLib.Alignment.Axioms.StrictlyPrefers ranking first second →
      AppliedModelingLib.Alignment.Axioms.linearReward parameter features second ≤
        AppliedModelingLib.Alignment.Axioms.linearReward parameter features first
```

Exact Lean library declaration

```
/- A parameter weakly respects every strict comparison in `ranking`. -/
def InducesRanking {n dimension : ℕ}
  (features : Candidate n → FeatureVector dimension)
  (parameter : LinearRewardParameter dimension) (ranking : Ranking n) : Prop :=
  ∀ first second,
    StrictlyPrefers ranking first second →
      linearReward parameter features second ≤ linearReward parameter features first
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening **Verdict:** matches

Reason: The source defines induction by a preferred-to-less-preferred pair having weakly larger reward; the Lean inequality $\text{reward}(\text{second}) \leq \text{reward}(\text{first})$ states the same condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.IsKemenySelector

Source locator: cited publication, lines 1423-1449

Verbatim paper-source connection

C.2 Linear Kemeny Rule

Next, we consider a different rule from social choice theory, the Kemeny rule, which can be transformed to the linear setting while maintaining the separability can be achieved along with PMC. Given an input profile π , the Kemeny rule returns a ranking σ^* that minimizes the total number of pairwise disagreements with voters' rankings, i.e.

$$\sigma^* \in \arg \min_{\sigma \in S} \sum_{i \in V} \sum_{(a,b): a \succ_i b} 1(b \succ \sigma a)$$

where S contains all possible permutations of the m candidates. This expression can be equivalently written as

$$\sigma^* \in \arg \min_{\sigma \in S} \sum_{a \succ b \in C} n_{a \succ b}(\pi) \cdot 1(b \succ \sigma a).$$

Here, we define the linear Kemeny rule, which outputs a parameter vector θ^* that induces a ranking that minimizes the total number of pairwise disagreements with voters' rankings, i.e.,

$$\theta^* \in \arg \min_{\theta \in R_d} \sum_{a \succ b \in C} n_{a \succ b}(\pi) \cdot 1(r_\theta(b) > r_\theta(a)).$$

Note that this rule conforms to the standard loss formulation, where the loss function is binary: it is 0 if two rankings agree with respect to the relative ranking of a pair of candidates and 1 otherwise. Since binary loss is not convex, it does not fit in the impossibility result of Theorem 3.1. Note that Kemeny is generally NP-hard to compute; however, even for linear Kemeny, there is at least an exponential time algorithm by brute-force computing the score of every ranking, and determining whether or not it is feasible.

Lean-expanded library semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
  ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
    AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer feasible profile (rule.run profile)
```

Exact Lean library declaration

```
-- A rule selects a Kemeny minimizer on every finite profile. -/
def IsKemenySelector {Voter : Type*} [Fintype Voter] {n : ℕ}
  (feasible : Ranking n → Prop) (rule : LinearRankAggregationRule Voter n feasible) : Prop :=
  ∀ profile, IsKemenyMinimizer feasible profile (rule.run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared the linear-Kemeny argmin model and the Theorem C.4 use. On every finite profile the expanded predicate requires the rule output to be feasible and to have no larger exact Kemeny disagreement count than any feasible contender, which is precisely minimization over linearly realizable rankings; it does not assume uniqueness or a hidden tie rule.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.NondegenerateParameter

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

```
type:
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → Prop

value:
fun {n dimension : ℕ}
(features :
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
∀ (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
first ≠ second →
AppliedModelingLib.Alignment.Axioms.linearReward parameter features first ≠
AppliedModelingLib.Alignment.Axioms.linearReward parameter features second
```

Exact Lean library declaration

```
-- A parameter gives distinct rewards to every pair of distinct candidates. -/
def NondegenerateParameter {n dimension : ℕ}
(features : Candidate n → FeatureVector dimension)
(parameter : LinearRewardParameter dimension) : Prop :=
∀ first second, first ≠ second →
linearReward parameter features first ≠ linearReward parameter features second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-library screening **Verdict:** matches

Reason: The source's nondegeneracy condition requires unequal rewards across candidate-separating comparisons; Lean states this for every distinct candidate pair.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking

Source locator: cited publication, lines 158-198

Verbatim paper-source connection

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_\theta : C \rightarrow R$ defined by taking the dot product with feature vectors $r_\theta(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a>b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a>b}$ and $R_{b>a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b:a \succ_\sigma b} R_{a>b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a>b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a>b}(\pi) = n_{a>b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a>b}$ and $w_{a>b}$, respectively. We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

Lean-expanded library semantic target

```
type:
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {n dimension : ℕ}
(features :
AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
∃ (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension),
AppliedModelingLib.Alignment.Axioms.NondegenerateParameter features parameter ∧
AppliedModelingLib.Alignment.Axioms.InducesRanking features parameter ranking
```

Exact Lean library declaration

```
/-- A ranking is feasible when some nondegenerate linear parameter induces it. -/
def LinearFeasibleRanking {n dimension : ℕ}
(features : Candidate n → FeatureVector dimension) (ranking : Ranking n) : Prop :=
∃ parameter : LinearRewardParameter dimension,
NondegenerateParameter features parameter ∧ InducesRanking features parameter ranking
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.InducesRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.NondegenerateParameter](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening **Verdict:** matches

Reason: The source calls a ranking feasible exactly when some nondegenerate parameter induces it, which is the existential conjunction in the declaration.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.MajorityConsistent

Source locator: cited publication, lines 441-443

Verbatim paper-source connection

Definition 4.1 (majority consistency). A linear rank aggregation rule satisfies majority consistency if when a candidate a is ranked first by a majority of voters in the input profile, a is ranked first in the output ranking.

Lean-expanded library semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
AppliedModelingLib.Alignment.Axioms.MajorityTopChoice profile candidate →
AppliedModelingLib.SocialChoice.Ranking.firstChoice (rule.run profile) = candidate
```

Exact Lean library declaration

```
/--
Definition 4.1 of Ge et al. (2024), specialized to finite ranking profiles.
-/
def MajorityConsistent {Voter : Type*} [Fintype Voter] {n : ℕ}
(feasible : Ranking n → Prop)
(rule : LinearRankAggregationRule Voter n feasible) : Prop :=
∀ profile candidate,
FeasibleProfile feasible profile → MajorityTopChoice profile candidate →
firstChoice (rule.run profile) = candidate
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.MajorityTopChoice](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [AppliedModelingLib.SocialChoice.Ranking.firstChoice](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared Definition 4.1 and its positive and negative theorem uses. After requiring a source-feasible profile, the target uses the exact strict-cardinality condition $\text{card}(V) < 2 * \text{topCount}$ and requires the aggregate first position to be that candidate, precisely ‘ranked first by a majority’ rather than by a mere plurality or weak half.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent

Source locator: cited publication, lines 204-207

Verbatim paper-source connection

Definition 2.2 (Pairwise Majority Consistency (PMC)). A ranking σ is called a PMC ranking for profile π if for all $a, b \in C$, $a \succ_{\sigma} b$ if and only if a majority of voters rank $a \succ_{\pi} b$, i.e., $w_a \succ b > 1/2$. A linear rank aggregation rule satisfies PMC if, when a PMC ranking σ exists for the input profile π and σ is feasible, then $f(\pi) = \sigma$.

Lean-expanded library semantic target

```
type:
{Voter : Type u_1} →
[FIntype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [FIntype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
(majorityRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
feasible majorityRanking →
AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile majorityRanking →
rule.run profile = majorityRanking
```

Exact Lean library declaration

```
/--
Definition 2.2 of Ge et al. (2024), specialized to finite ranking profiles:
when a feasible ranking realizes all strict pairwise majorities, the rule
returns that ranking.
-/
def PairwiseMajorityConsistent {Voter : Type*} [FIntype Voter] {n : ℕ}
(feasible : Ranking n → Prop)
(rule : LinearRankAggregationRule Voter n feasible) : Prop :=
∀ profile majorityRanking,
FeasibleProfile feasible profile → feasible majorityRanking →
IsPairwiseMajorityRanking profile majorityRanking → rule.run profile = majorityRanking
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Definition 2.2 requires a rule to return a feasible ranking whenever that ranking realizes every strict pairwise-majority comparison. The declaration quantifies over exactly those feasible profiles and rankings and requires the rule output to equal that ranking.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.ParetoOptimal

Source locator: cited publication, lines 201-203

Verbatim paper-source connection

Definition 2.1 (Pareto Optimality). A linear rank aggregation rule f satisfies Pareto optimality if, whenever every voter prefers candidate a over candidate b on π , i.e., $w_{a>b}(\pi) = 1$, then candidate a is ranked higher than candidate b in the output ranking, i.e., $a >_f(\pi) b$.

Lean-expanded library semantic target

```
type:
  {Voter : Type u_1} →
  {n : ℕ} →
    (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
  fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
    (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
    ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
      (first second : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
      AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile →
      AppliedModelingLib.Alignment.Axioms.UniversallyPreferred profile first second →
      AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (rule.run profile) first second
```

Exact Lean library declaration

```
/--
Definition 2.1 of Ge et al. (2024), specialized to finite ranking profiles:
unanimously preferred candidates remain ordered that way by the rule.
-/
def ParetoOptimal {Voter : Type*} {n : ℕ} (feasible : Ranking n → Prop)
  (rule : LinearRankAggregationRule Voter n feasible) : Prop :=
  ∀ profile first second,
    FeasibleProfile feasible profile → UniversallyPreferred profile first second →
    StrictlyPrefers (rule.run profile) first second
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.UniversallyPreferred](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared Definition 2.1 and all PO success/failure uses. For every source-feasible profile and candidate pair, unanimous strict preference by all voters implies the same strict comparison in the output ranking, exactly the $w_{a>b}=1$ condition and conclusion; no majority or score surrogate replaces unanimity.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.RankingSeparability

Source locator: cited publication, lines 1389-1391

Verbatim paper-source connection

Definition C.1 (Separability). A ranking aggregation rule satisfies ranking separability (or separability for short) if, when two profiles yield identical output rankings, when combined into a single profile, this should also produce the same output ranking.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
((voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible) →
Prop

value:
fun {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (rule :
    (voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible) =>
  ∀ (leftCount rightCount : ℕ) (left : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin leftCount) n)
    (right : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin rightCount) n),
    (rule leftCount).run left = (rule rightCount).run right →
    (rule (leftCount + rightCount)).run (AppliedModelingLib.Alignment.Axioms.rankingProfileAppend left right) =
    (rule leftCount).run left
```

Exact Lean library declaration

```
/-
Definition C.1 of Ge et al. (2024), with a rule family indexed by the finite
number of voters. The conclusion compares the two original equal outputs to
the output on their literal concatenation.
-/
def RankingSeparability {n : ℕ} (feasible : Ranking n → Prop)
  (rule : ∀ voterCount : ℕ, LinearRankAggregationRule (Fin voterCount) n feasible) : Prop :=
  ∀ leftCount rightCount
    (left : RankingProfile (Fin leftCount) n)
    (right : RankingProfile (Fin rightCount) n),
    (rule leftCount).run left = (rule rightCount).run right →
    (rule (leftCount + rightCount)).run (rankingProfileAppend left right) =
    (rule leftCount).run left
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.rankingProfileAppend](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared Definition C.1 and all four separability uses. The target quantifies two finite profiles, assumes their selected output rankings are identical, forms their literal disjoint concatenation with left and right voter counts, and requires the combined output to equal that common ranking. This is exactly the source property, including exact ranking equality rather than score equivalence.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.WinnerMonotonic

Source locator: cited publication, lines 452-454

Verbatim paper-source connection

Definition 4.2 (winner monotonicity). A linear rank aggregation rule satisfies winner monotonicity if, when a candidate a is ranked first in the output ranking, elevating a in any voter's preference does not cause a to lose their top position in the updated aggregate ranking.

Lean-expanded library semantic target

```
type:
  {Voter : Type u_1} →
  {n : ℕ} →
    (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
  fun {Voter : Type u_1} {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
    (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
    ∀ (original updated : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (voter : Voter)
      (candidate : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
      AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible original →
      AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible updated →
      AppliedModelingLib.SocialChoice.Ranking.firstChoice (rule.run original) = candidate →
      AppliedModelingLib.Alignment.Axioms.ProfileElevatesCandidate original updated voter candidate →
      AppliedModelingLib.SocialChoice.Ranking.firstChoice (rule.run updated) = candidate
```

Exact Lean library declaration

```
/--
Definition 4.2 of Ge et al. (2024), specialized to feasible finite profiles.
Elevating an output winner in one submitted ranking leaves it the output
winner after the update.
-/
def WinnerMonotonic {Voter : Type*} {n : ℕ} (feasible : Ranking n → Prop)
  (rule : LinearRankAggregationRule Voter n feasible) : Prop :=
  ∀ original updated voter candidate,
    FeasibleProfile feasible original → FeasibleProfile feasible updated →
    firstChoice (rule.run original) = candidate →
    ProfileElevatesCandidate original updated voter candidate →
    firstChoice (rule.run updated) = candidate
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.ProfileElevatesCandidate](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [AppliedModelingLib.SocialChoice.Ranking.firstChoice](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared Definition 4.2 and its Theorems 4.3/C.6 uses. The target changes exactly one voter, weakly raises the named output winner's position, preserves the relative order of every other candidate and every other voter's ranking, requires both profiles source-feasible, and concludes that the same candidate remains first. This is the source elevation operation and winner conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.canonicalKemenyRule

Source locator: cited publication, lines 1423-1449

Verbatim paper-source connection

C.2 Linear Kemeny Rule

Next, we consider a different rule from social choice theory, the Kemeny rule, which can be transformed to the linear setting while maintaining the separability can be achieved along with PMC. Given an input profile π , the Kemeny rule returns a ranking σ^* that minimizes the total number of pairwise disagreements with voters' rankings, i.e.

$$\sigma^* \in \arg \min_{\sigma \in S^m} \sum_{i \in V(a,b): a >_{\sigma} b} 1(b >_{\sigma} a)$$

where S contains all possible permutations of the m candidates. This expression can be equivalently written as

$$\sigma^* \in \arg \min_{\sigma \in S^m} \sum_{a > b \in C} n_{a > b}(\pi) \cdot 1(b >_{\sigma} a).$$

Here, we define the linear Kemeny rule, which outputs a parameter vector θ^* that induces a ranking that minimizes the total number of pairwise disagreements with voters' rankings, i.e.,

$$\theta^* \in \arg \min_{\theta \in R^d} \sum_{a > b \in C} n_{a > b}(\pi) \cdot 1(r_{\theta}(b) > r_{\theta}(a)).$$

Note that this rule conforms to the standard loss formulation, where the loss function is binary: it is 0 if two rankings agree with respect to the relative ranking of a pair of candidates and 1 otherwise. Since binary loss is not convex, it does not fit in the impossibility result of Theorem 3.1.

Note that Kemeny is generally NP-hard to compute; however, even for linear Kemeny, there is at least an exponential time algorithm by brute-force computing the score of every ranking, and determining whether or not it is feasible.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
(fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n →
feasible fallback →
(voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible
value:
fun {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (hfallback : feasible fallback) (voterCount : ℕ) =>
{
run := fun (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin voterCount) n) =>
↑ (AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection feasible fallback hfallback profile),
output_feasible :=
AppliedModelingLib.Alignment.Axioms.canonicalKemenyRule._proof_1 feasible fallback hfallback voterCount }
```

Exact Lean library declaration

```
/-- The canonical finite Kemeny rule. -/
noncomputable def canonicalKemenyRule {n : ℕ}
(feasible : Ranking n → Prop) (fallback : Ranking n) (hfallback : feasible fallback)
(voterCount : ℕ) : LinearRankAggregationRule (Fin voterCount) n feasible where
run profile := (canonicalKemenySelection feasible fallback hfallback profile).val
output_feasible profile :=
(canonicalKemenySelection feasible fallback hfallback profile).property.1.1
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.IsKemenyMinimizer](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.canonicalKemenySelection](#)
- [AppliedModelingLib.Alignment.Axioms.rankingTieKey](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared the linear-Kemeny rule model and Theorem C.3. The expanded rule chooses a feasible ranking minimizing the exact disagreement objective and then applies one profile-independent finite ranking key only among co-minimizers; this is a deterministic choice permitted by the source argmin, and the fixed key supplies the consistent selection used by the additive separability argument.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Alignment.Axioms.kemenyDisagreement

Source locator: cited publication, lines 1423-1449

Verbatim paper-source connection

C.2 Linear Kemeny Rule

Next, we consider a different rule from social choice theory, the Kemeny rule, which can be transformed to the linear setting while maintaining the separability can be achieved along with PMC. Given an input profile π , the Kemeny rule returns a ranking σ^* that minimizes the total number of pairwise disagreements with voters rankings, i.e.

$$\sigma^* \in \arg \min_{\sigma \in S} \sum_{i \in V} \sum_{a >_{\sigma} b} 1(b >_{\pi_i} a)$$

where S contains all possible permutations of the m candidates. This expression can be equivalently written as

$$\sigma^* \in \arg \min_{\sigma \in S} \sum_{a >_{\pi} b} n_{a > b}(\pi) \cdot 1(b >_{\sigma} a).$$

Here, we define the linear Kemeny rule, which outputs a parameter vector θ^* that induces a ranking that minimizes the total number of pairwise disagreements with voters' rankings, i.e.,

$$\theta^* \in \arg \min_{\theta \in R_d} \sum_{a >_{\pi} b} n_{a > b}(\pi) \cdot 1(r_{\theta}(b) > r_{\theta}(a)).$$

Note that this rule conforms to the standard loss formulation, where the loss function is binary: it is 0 if two rankings agree with respect to the relative ranking of a pair of candidates and 1 otherwise. Since binary loss is not convex, it does not fit in the impossibility result of Theorem 3.1.

Note that Kemeny is generally NP-hard to compute; however, even for linear Kemeny, there is at least an exponential time algorithm by brute-force computing the score of every ranking, and determining whether or not it is feasible.

Lean-expanded library semantic target

```

type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℕ

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (output : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  ∑ voter : Voter,
  ∑ first : AppliedModelingLib.SocialChoice.Ranking.Candidate n,
  ∑ second : AppliedModelingLib.SocialChoice.Ranking.Candidate n,
  if
    AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (profile voter) first second ∧
    AppliedModelingLib.Alignment.Axioms.StrictlyPrefers output second first then
  1
  else 0

```

Exact Lean library declaration

```

/--
The Kemeny disagreement count of an output ranking against a finite profile.
For every ordered candidate pair submitted in one voter's ranking, it charges
one exactly when the output reverses that pair. This is the source's
`n_{a > b}(\pi) * 1(b >_{\sigma} a)` objective written as a direct voter sum.
-/
def kemenyDisagreement {Voter : Type*} [Fintype Voter] {n : ℕ}
  (profile : RankingProfile Voter n) (output : Ranking n) : ℕ :=
  ∑ voter, ∑ first, ∑ second,
  if StrictlyPrefers (profile voter) first second ∧ StrictlyPrefers output second first then 1
  else 0

```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared both displayed Kemeny formulas and all C.3-C.5 uses. The target sums over voters and ordered candidate pairs and charges one exactly when the voter ranks first above second while the output ranks second above first. Regrouping by pair yields $n_{a > b}(\pi) * 1(b >_{\sigma} a)$, with self-pairs contributing zero, exactly the source disagreement count.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Paper-specific semantic prerequisites

GeEtAI2024AlignmentAxioms.IsLeximaxPluralitySelector

Paper-local declaration:

GeEtAI2024AlignmentAxioms.IsLeximaxPluralitySelector

Source locator: cited publication, lines 1530-1539

Verbatim paper-source connection

C.3 Leximax Plurality

Plurality is probably the most ubiquitous voting rule in the world. Its ranking variant ranks the candidates in decreasing order with respect to their plurality scores. The plurality score of a candidate is equal to the number of her appearances in the first position. This rule is known to satisfy several axioms but in linear social choice cannot be directly applied, as not all rankings are feasibly. Similarly to leximax Copeland, we define Leximax Plurality as follows. It ranks first the candidate with the highest plurality score that can be ranked first under some parameter vector. Subject to this first position, it ranks second the candidate with the highest plurality score that can feasibly be ranked second, and so on, until all the positions are filled.

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible) =>
  ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
    GeEtAI2024AlignmentAxioms.IsLeximaxPluralityOutcome feasible profile (rule.run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAI2024AlignmentAxioms.IsLeximaxPluralityOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded predicate requires the output to be feasible and, at every sequential prefix, forbids a feasibly placeable next candidate with a strictly higher plurality score. This is precisely the source's sequential highest-feasible-plurality prescription, before any tie determinization.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAI2024AlignmentAxioms.majorityLoss

Paper-local declaration:

GeEtAI2024AlignmentAxioms.majorityLoss

Source locator: cited publication, lines 370-386

Verbatim paper-source connection

3.2 Majority-Based Loss Formulation

Despite the negative results for loss-function-based rules, we may hope for a remedy using slightly different information. Specifically, we consider a similar loss-based function that rather than getting penalized for disagreeing with each voter only gets penalized if it disagrees with a majority of voters. That is, we choose θ minimizing

$$L_{maj}(\theta; \pi, ell) = \sum_{a,b \in C} I[wa > b(\pi) > 1/2] \cdot ell(r\theta(b) - r\theta(a)).$$

Defining a parameter aggregation function based on this loss suffers from the same caveat as before, that in some cases no optimal θ exists. Nevertheless, we can apply an analogous fix for a ranking variant. We say that a linear rank aggregation rule f minimizes ell in the majority formulation if for all $\sigma = f(\pi)$,

$$\inf_{\theta: \theta \in R} \sigma L_{maj}(\theta; \pi, ell) = \inf_{\theta} L_{maj}(\theta; \pi, ell).$$

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
 AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
(ℝ → ℝ) → AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
(features :
 AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ)
(parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
Σ pair : AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n,
if AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers profile pair.1 pair.2 then
  loss
  (AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.2 -
   AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.1)
else 0
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictMajorityPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictMajorityPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded objective sums a loss term exactly when a strict voter majority ranks the ordered pair first-over-second, at the reward difference second minus first. This is the routed formula for L_{maj} ; self-pairs contribute zero because strict self-preference is false.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAl2024AlignmentAxioms.globalMajorityLossInfimum

Paper-local declaration:

GeEtAl2024AlignmentAxioms.globalMajorityLossInfimum

Source locator: cited publication, lines 370-386

Verbatim paper-source connection

3.2 Majority-Based Loss Formulation

Despite the negative results for loss-function-based rules, we may hope for a remedy using slightly different information. Specifically, we consider a similar loss-based function that rather than getting penalized for disagreeing with each voter only gets penalized if it disagrees with a majority of voters. That is, we choose θ minimizing

$$L_{\text{maj}}(\theta; \pi, \text{ell}) = \sum_{a,b \in C} I[w_{a>b}(\pi) > 1/2] \cdot \text{ell}(r_{\theta}(b) - r_{\theta}(a)).$$

Defining a parameter aggregation function based on this loss suffers from the same caveat as before, that in some cases no optimal θ exists. Nevertheless, we can apply an analogous fix for a ranking variant. We say that a linear rank aggregation rule f minimizes ell in the majority formulation if for all $\sigma = f(\pi)$,

$$\inf_{\theta: \theta \in R} \sigma L_{\text{maj}}(\theta; \pi, \text{ell}) = \inf_{\theta} L_{\text{maj}}(\theta; \pi, \text{ell}).$$

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
 AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n → (ℝ → ℝ) → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ) =>
  sInf (Set.range (GeEtAl2024AlignmentAxioms.majorityLoss features profile loss))
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.majorityLoss](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The declaration is 'sInf' of the range of the expanded majority-loss objective over all linear reward parameters, exactly the unrestricted infimum in the routed majority-loss formulation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAl2024AlignmentAxioms.restrictedMajorityLossInfimum

Paper-local declaration:

GeEtAl2024AlignmentAxioms.restrictedMajorityLossInfimum

Source locator: cited publication, lines 370-386

Verbatim paper-source connection

3.2 Majority-Based Loss Formulation

Despite the negative results for loss-function-based rules, we may hope for a remedy using slightly different information. Specifically, we consider a similar loss-based function that rather than getting penalized for disagreeing with each voter only gets penalized if it disagrees with a majority of voters. That is, we choose θ minimizing

$$L_{\text{maj}}(\theta; \pi, \text{ell}) = \sum_{a,b \in C} I[\text{wa} > \text{b}(\pi) > 1/2] \cdot \text{ell}(r\theta(b) - r\theta(a)).$$

Defining a parameter aggregation function based on this loss suffers from the same caveat as before, that in some cases no optimal θ exists. Nevertheless, we can apply an analogous fix for a ranking variant. We say that a linear rank aggregation rule f minimizes ell in the majority formulation if for all $\sigma = f(\pi)$,

$$\inf_{\theta: \theta \in R} \sigma L_{\text{maj}}(\theta; \pi, \text{ell}) = \inf_{\theta} L_{\text{maj}}(\theta; \pi, \text{ell}).$$

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n →
 AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
(ℝ → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
(features :
 AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) (loss : ℝ → ℝ)
(ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
sInf
(GeEtAl2024AlignmentAxioms.majorityLoss features profile loss "
 GeEtAl2024AlignmentAxioms.inducingParameters features ranking)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.inducingParameters](#)
- [GeEtAl2024AlignmentAxioms.majorityLoss](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The declaration takes ‘sInf’ of the majority-loss image of precisely the parameters that induce the supplied ranking. This is the source’s infimum restricted to the parameter region R^{σ} .

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing

Paper-local declaration:

GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing

Source locator: cited publication, lines 370-386

Verbatim paper-source connection

3.2 Majority-Based Loss Formulation

Despite the negative results for loss-function-based rules, we may hope for a remedy using slightly different information. Specifically, we consider a similar loss-based function that rather than getting penalized for disagreeing with each voter only gets penalized if it disagrees with a majority of voters. That is, we choose θ minimizing

$$L_{\text{maj}}(\theta; \pi, \text{ell}) = \sum_{a,b \in C} \mathbb{I}[w_{a>b}(\pi) > 1/2] \cdot \text{ell}(r_{\theta}(b) - r_{\theta}(a)).$$

Defining a parameter aggregation function based on this loss suffers from the same caveat as before, that in some cases no optimal θ exists. Nevertheless, we can apply an analogous fix for a ranking variant. We say that a linear rank aggregation rule f minimizes ell in the majority formulation if for all $\sigma = f(\pi)$,

$$\inf_{\theta: \theta \in R} \sigma L_{\text{maj}}(\theta; \pi, \text{ell}) = \inf_{\theta} L_{\text{maj}}(\theta; \pi, \text{ell}).$$

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n dimension : ℕ} →
(features :
  AppliedModelingLib.SocialChoice.Ranking.Candidate n →
  AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
(ℝ → ℝ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) →
  Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (loss : ℝ → ℝ)
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features)) =>
  ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
  AppliedModelingLib.Alignment.Axioms.FeasibleProfile
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) profile →
  GeEtAl2024AlignmentAxioms.restrictedMajorityLossInfimum features profile loss (rule.run profile) =
  GeEtAl2024AlignmentAxioms.globalMajorityLossInfimum features profile loss
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.globalMajorityLossInfimum](#)
- [GeEtAl2024AlignmentAxioms.restrictedMajorityLossInfimum](#)

Recorded source-to-declaration screening Verdict: matches

Reason: For every feasible profile, the expanded target equates the infimum of the majority-loss objective over parameters inducing the rule output with its unrestricted infimum. This is the source's definition of a rule minimizing the majority formulation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAl2024AlignmentAxioms.IsParetoKemenySelector

Paper-local declaration:

GeEtAl2024AlignmentAxioms.IsParetoKemenySelector

Source locator: cited publication, lines 1506-1508

Verbatim paper-source connection

A possibly easy fix to the problem that linear Kemeny does not satisfy PO would be to enforce the PO criterion, as we did for the LCPO, i.e., to restrict to parameter vectors that respect PO. However, we show that if we do that, then linear Kemeny subject to PO does not satisfy separability anymore.

Lean-expanded paper semantic target

```
type:
((voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 18
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)) →
Prop

value:
fun
  (rule :
    (voterCount : ℕ) →
      AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 18
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)) =>
  ∀ (voterCount : ℕ) (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile (Fin voterCount) 18),
    AppliedModelingLib.Alignment.Axioms.FeasibleProfile
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) profile →
    GeEtAl2024AlignmentAxioms.IsParetoKemenyMinimizer profile ((rule voterCount).run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [GeEtAl2024AlignmentAxioms.IsParetoKemenyMinimizer](#)
- [GeEtAl2024AlignmentAxioms.c4Features](#)

Recorded source-to-declaration screening Verdict: matches

Reason: For the displayed source feature instance, the expanded target requires the output to be feasible, to respect every unanimous comparison, and to minimize Kemeny disagreement among all such rankings on every feasible profile. This is exactly Kemeny restricted to PO-respecting parameter-induced rankings; it adds no global tie rule.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralitySelector

Source locator: cited publication, lines 1530-1539

Verbatim paper-source connection

C.3 Leximax Plurality

Plurality is probably the most ubiquitous voting rule in the world. Its ranking variant ranks the candidates in decreasing order with respect to their plurality scores. The plurality score of a candidate is equal to the number of her appearances in the first position. This rule is known to satisfy several axioms but in linear social choice cannot be directly applied, as not all rankings are feasibly. Similarly to leximax Copeland, we define Leximax Plurality as follows. It ranks first the candidate with the highest plurality score that can be ranked first under some parameter vector. Subject to this first position, it ranks second the candidate with the highest plurality score that can feasibly be ranked second, and so on, until all the positions are filled.

[Next verbatim source excerpt]

[form-feed in source extraction]

Theorem C.6. Leximax Plurality satisfies majority consistency, winner monotonicity and separability.

Proof. Note that leximax Plurality always returns a ranking in which the candidate with the highest plurality score is ranked first, since there exists at least one feasible ranking in which this candidate is ranked first. From this observation, we immediately see that leximax Plurality satisfies majority consistency.

Now, suppose that on input profile π , the rule outputs a ranking such that candidate a is ranked first. This means that a has the highest plurality score. Now, consider a profile π' which is similar to π with the only exception being a ranking in which a is ranked in a higher position. It is clear that a continues to have the highest plurality score and therefore leximax Plurality will output a ranking in which a is ranked first. Therefore, winner monotonicity is satisfied.

It remains to prove that the rule satisfies separability. Suppose that under two different profiles π_1 and π_2 , the rule outputs σ , and under the aggregated profile π_3 , it outputs σ' . We will show that $\sigma = \sigma'$. One main observation is that if a candidate a has a higher plurality score than a candidate b under both π_1 and π_2 , then a has a higher plurality score than b under π_3 as well. We will show the desired property by induction on the positions of the ranking σ . Start from the first position in which say candidate a is ranked first. From above, we know that a has the highest plurality score under both π_1 and π_2 , which remains true in π_3 , and therefore a is ranked first in σ' . Now, assume that up to position $t - 1$, σ and σ' are similar and denote with a' the candidate that is ranked at position t of σ . We denote by S_1 , S_2 and S_3 the set of candidates that are not ranked among the first t positions in σ and have higher plurality score than a' under π_1 , π_2 , and π_3 respectively. Since, a' is ranked at the t -th position in σ , we get that, subject to the fixed first $t - 1$ position, no candidate in $S_1 \cup S_2$ can be ranked at the t -th position. The theorem follows by noticing that $S_3 \subseteq S_1 \cup S_2$, which follows from the main observation above.

27

Settled source reading The checked C.6 rule uses one fixed profile-independent injective candidate priority for equal plurality scores. Strict plurality comparisons are unchanged; the theorem has this specified rule-family scope.

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[Fintype Voter] →
{n : ℕ} →
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible →
(AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) → Prop

value:
fun {Voter : Type u_1} [Fintype Voter] {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ) =>
Function.Injective priority ∧
∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralityOutcome feasible priority profile (rule.run profile)
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralityOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The raw C.6 text leaves equal plurality scores unresolved. Under the attached approved GE24-C6-FIXED-TIE-CONVENTION-01 clarification, the declaration gives one injective profile-independent priority that is consulted only at equal score and therefore preserves every strict source plurality comparison.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

GeEtAl2024AlignmentAxioms.fixedTieLCPO

Paper-local declaration:

GeEtAl2024AlignmentAxioms.fixedTieLCPO

Source locator: cited publication, lines 416-438

Verbatim paper-source connection

The Copeland rule assigns a Copeland score to each alternative equal to the number of other alternatives it beats in a pairwise competition, i.e., the score for a is $|\{b \mid w_a > b > 1/2\}|$. It then ranks the candidates in descending order according to their Copeland scores (breaking ties arbitrarily). It

7

[form-feed in source extraction]

is known that Copeland satisfies PO, PMC, and additional axiomatic properties. However, in linear social choice, since not every ranking is feasible, we cannot always output the Copeland ranking. We, therefore, define a new linear rank aggregation rule, which we call leximax Copeland. This rule chooses a feasible ranking as follows. It ranks first the candidate with the highest Copeland score that can be feasibly ranked first under some parameter vector θ . Subject to this first position, it ranks second the candidate with the highest Copeland score which can be feasibly ranked second, and continues this process for subsequent positions.

Copeland's rule is a C1 rule because it only requires the majority relationships between the candidates. Analogously, leximax Copeland is also a C1 linear rank aggregation rule. Therefore, by Theorem 3.7, it does not satisfy the PO criterion. To address this issue, we define a variant called leximax Copeland subject to PO (LCPO), which incorporates the PO criterion. Under LCPO, for every pair of alternatives where one dominates the other, the rule restricts rankings to place the dominating alternative above the dominated one.

The rule remains well-defined since the set of feasible rankings when enforcing the PO criterion is non-empty, as whenever a dominates b , all the rankings in the input profile rank a above b . Note that if the Copeland ranking is feasible, then this rule outputs that ranking, since unrestricted Copeland satisfies PO.

Settled source reading The checked Theorem 4.3 LCPO rule uses one fixed profile-independent injective candidate priority for equal Copeland scores. This is a declared source clarification, not an inference from single-valuedness.

Lean-expanded paper semantic target

```
type:
  {Voter : Type u_1} →
  [Fintype Voter] →
  [Nonempty Voter] →
  {n : ℕ} →
  (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop) →
  (fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
  feasible fallback → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible

value:
fun {Voter : Type u_1} [Fintype Voter] [Nonempty Voter] {n : ℕ}
  (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
  (fallback : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (hfallback : feasible fallback) =>
  {
    run := fun (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n) =>
      if hprofile : AppliedModelingLib.Alignment.Axioms.FeasibleProfile feasible profile then
        GeEtAl2024AlignmentAxioms.fixedTieLCPOOutcome feasible profile hprofile
      else fallback,
    output_feasible := GeEtAl2024AlignmentAxioms.fixedTieLCPO._proof_1 feasible fallback hfallback }
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.fixedTieLCPOOutcome](#)

Recorded source-to-declaration screening Verdict: matches

Reason: On source-feasible profiles the expanded run searches rankings that are both feasible and Pareto-respecting, then lexicographically maximizes the positionwise Copeland keys. This implements the source's LCPO restriction and sequential highest-Copeland selection; its candidate-order tie break instantiates the source's arbitrary ties.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GeEtAI2024AlignmentAxioms.standardLossFormulationSpec

Paper-local declaration:

GeEtAI2024AlignmentAxioms.standardLossFormulationSpec

Source locator: cited publication, lines 237-259

Verbatim paper-source connection

3 Loss-Based Rules
3.1 Standard Loss Formulation

We begin our study of linear social choice by considering a quite broad yet natural class of rules that capture how RLHF is currently being done. Their core idea is the following: when considering parameter vector θ , for each voter i that ranks a pair of candidates $a \succ_i b$, we should incur some loss for giving b a higher reward than a . To formalize this, let $\text{ell} : \mathbb{R} \rightarrow \mathbb{R}$ be a loss function, which we assume is nonnegative. We can then choose a parameter vector minimizing

$$L(\theta; \pi, \text{ell}) = \sum_{a \succ b \in C} \pi(a) \cdot \text{ell}(r_\theta(b) - r_\theta(a)).$$

Note that the BTL model fits within this framework using $\text{ell}(x) = \ln(1 + e^x)$, i.e., binary cross-entropy loss. One caveat to this approach, however, is that an optimal θ need not be well-defined: it is possible that no minimum is attained. Fortunately, since we only care about rankings induced by optimal parameter vectors, we can conveniently remedy this by saying the output is any ranking that is induced by parameter vectors that are arbitrarily close to optimal. More formally, we say that a linear rank aggregation rule f minimizes ell if for all $\sigma = f(\pi)$,

$$\inf_{\theta \in \mathbb{R}^\sigma} L(\theta; \pi, \text{ell}) = \inf_{\theta} L(\theta; \pi, \text{ell}). \quad (1)$$

Even if no minimum is attained, there is always a choice of feasible ranking σ such that Equation (1) is satisfied.

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
[FIntype Voter] →
{n dimension : ℕ} →
(features :
  AppliedModelingLib.SocialChoice.Ranking.Candidate n →
  AppliedModelingLib.Alignment.Axioms.FeatureVector dimension) →
(ℝ → ℝ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) →
  Prop

value:
fun {Voter : Type u_1} [FIntype Voter] {n dimension : ℕ}
  (features :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
  (loss : ℝ → ℝ)
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features)) =>
have objective :
  AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n →
  AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension → ℝ :=
fun (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension) =>
  ∑ voter : Voter,
  ∑ pair :
    AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n,
    if AppliedModelingLib.Alignment.Axioms.StrictlyPrefers (profile voter) pair.1 pair.2 then
      loss
        (AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.2 -
         AppliedModelingLib.Alignment.Axioms.linearReward parameter features pair.1)
    else 0;
  (∀ (input : ℝ), 0 ≤ loss input) ∧
  ∀ (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n),
  AppliedModelingLib.Alignment.Axioms.FeasibleProfile
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) profile →
  snf
    (objective profile "
      {parameter : AppliedModelingLib.Alignment.Axioms.LinearRewardParameter dimension |
        AppliedModelingLib.Alignment.Axioms.InducesRanking features parameter (rule.run profile)}) =
    snf (Set.range (objective profile))
```

Direct retained dependencies

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)
- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.InducesRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRewardParameter](#)

- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.Alignment.Axioms.StrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.instDecidableStrictlyPrefers](#)
- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded target requires a nonnegative loss, sums loss over every strict submitted comparison at $r_{\theta}(b) - r_{\theta}(a)$, and equates the infimum restricted to parameters inducing the output ranking with the unrestricted infimum. These are the routed standard-loss formulation's stated components.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text results

Review row 1

Source locator: cited publication, lines 267-269

Verbatim source input

Theorem 3.1. If a linear rank aggregation rule f optimizes a loss function ℓ that satisfies $\inf_x \ell(x) < \ell(0)$ and is either nondecreasing and weakly convex, or strictly convex (and possibly nonmonotone), then f fails PMC and PO.

[Next verbatim source excerpt]

[form-feed in source extraction]

3 Loss-Based Rules
3.1 Standard Loss Formulation

We begin our study of linear social choice by considering a quite broad yet natural class of rules that capture how RLHF is currently being done. Their core idea is the following: when considering parameter vector θ , for each voter i that ranks a pair of candidates $a \succ_i b$, we should incur some loss for giving b a higher reward than a . To formalize this, let $\ell : \mathbb{R} \rightarrow \mathbb{R}$ be a loss function, which we assume is nonnegative. We can then choose a parameter vector minimizing

$$L(\theta; \pi, \ell) = \sum_{a \succ b \in C} \pi(a, b) \cdot \ell(r_\theta(b) - r_\theta(a)).$$

Note that the BTL model fits within this framework using $\ell(x) = \ln(1 + e^x)$, i.e., binary cross-entropy loss.⁵ One caveat to this approach, however, is that an optimal θ need not be well-defined: it is possible that no minimum is attained. Fortunately, since we only care about rankings induced by optimal parameter vectors, we can conveniently remedy this by saying the output is any ranking that is induced by parameter vectors that are arbitrarily close to optimal. More formally, we say that a linear rank aggregation rule f minimizes ℓ if for all $\sigma = f(\pi)$,

$$\inf_{\theta \in \mathbb{R}^d} L(\theta; \pi, \ell) = \inf_{\sigma} L(\sigma; \pi, \ell). \quad (1)$$

Even if no minimum is attained, there is always a choice of feasible ranking σ such that Equation (1) is satisfied.

With this definition in hand, we proceed to our first main result, which spells rather bad news for this class of rules: Any loss-based aggregation rule using a nondecreasing and convex loss function (of which BTL is one, and hinge loss is another) will fail our two core axioms, PMC and PO. This paints a negative picture for current RLHF methods with respect to their social choice guarantees. Note that we will exclude the discussion of loss functions with a global minimum at zero, like ReLU, because the loss minimizer will be zero, making all rankings vacuous consequently. And we have focused on convex loss functions due to their practical optimization ease.

Theorem 3.1. If a linear rank aggregation rule f optimizes a loss function ℓ that satisfies $\inf_x \ell(x) < \ell(0)$ and is either nondecreasing and weakly convex, or strictly convex (and possibly nonmonotone), then f fails PMC and PO.

[Next verbatim source excerpt]

Proof. Fix a loss function ℓ satisfying the theorem conditions. Note that since ℓ is convex, we may also assume it is continuous [24, Corollary 10.1.1]. Furthermore, since $\inf_x \ell(x) < \ell(0)$, we know that there exists $x^* = 0$ such that $\ell(x) < \ell(0)$. The case where $x > 0$ is relatively simple (as such loss functions lead to unnatural behavior), and we handle it at the end of the proof. For now, we assume that there exists $x < 0$ such that $\ell(x) < \ell(0)$. Note that this also implies that for all $y \geq 0$, ℓ is lower bounded by the affine linear function connecting $(x, \ell(x))$ and $(0, \ell(0))$, and thus, $\lim_{x \rightarrow \infty} \ell(x) = \infty$.

We begin with a small instance of just three candidates $C = \{a, b, c\}$ to gain some traction on how ℓ behaves. We will later extend this instance with additional candidates to demonstrate a profile where PO and PMC fail. The candidates will have feature vectors $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. Furthermore, a p -fraction of voters (for p to be chosen later) will rank $a \succ b \succ c$, while the remaining $(1 - p)$ -fraction will have inverted preferences, ranking $c \succ b \succ a$.⁶ Let

$$L_{\text{core}}(\theta) := \sum_{a \succ b \in C} \pi(a, b) \cdot \ell(r_\theta(b) - r_\theta(a))$$

⁵ Typically, BTL is presented as choosing θ maximizing the likelihood of seeing the pairwise comparisons we observed, assuming that $\Pr[a \succ b] = \frac{e^{r_\theta(a)}}{e^{r_\theta(a)} + e^{r_\theta(b)}}$. That is, we choose θ maximizing

$$\sum_{a \succ b} \pi(a, b) \cdot \log \left(\frac{e^{r_\theta(a)}}{e^{r_\theta(a)} + e^{r_\theta(b)}} \right)$$

⁶ By taking the log and swapping the sign, we see that this is equivalent to minimizing $\sum_{a \succ b} \pi(a, b) \cdot (r_\theta(b) - r_\theta(a))$.
minimizing $\sum_{a \succ b} \log(1 + e^{r_\theta(b) - r_\theta(a)})$.

⁶ It so happens that these rankings are feasible, but for now we will not worry about this as loss function-based rules still make sense regardless of whether the inputs are feasible. For the final example, we will ensure that the rankings are feasible.

5

[form-feed in source extraction]

with $w_{x \succ y} \in \{1 - p, p\}$ be the loss function on this instance (scaling $n_{x \succ y}$ down to $w_{x \succ y}$ leads to an equivalent formulation). Let $g(x) = p \cdot \ell(-x) + (1 - p)\ell(x)$. Note that we can rewrite L_{core} as

$$L_{\text{core}}(\theta) = g(r_\theta(a) - r_\theta(b)) + g(r_\theta(a) - r_\theta(c)) + g(r_\theta(b) - r_\theta(c)).$$

Note that $r_\theta(c) = 0$ for all θ , so we can simplify this to

$$L_{\text{core}}(\theta) = g(r_\theta(a) - r_\theta(b)) + g(r_\theta(a)) + g(r_\theta(b)).$$

We will consider an unconstrained version of this problem where we are free to choose rewards $r_a, r_b \in \mathbb{R}$ arbitrarily, and later show by which vectors θ these optimal values can be induced. That

is, we will first find $r_a, r_b \in R$ minimizing $L_{unconstr}(r_a, r_b) := g(r_a - r_b) + g(r_a) + g(r_b)$.
Let $OP\ T\ core = \{\theta \mid L_{core}(\theta) = \inf_{\theta'} L_{core}(\theta')\}$ and $OP\ T\ unconstr = \{(r_a, r_b) \mid L_{unconstr}(r_a, r_b) = \inf_{r'_a, r'_b} L_{unconstr}(r'_a, r'_b)\}$ be the set of minimizers for these two loss functions. In Appendix A, we establish the following results about these optimal sets.
Lemma 3.2. There exists a rational $p \in (1/2, 1]$ and values $A_1 < A_2$ with $A_2 > 0$ such that $OP\ T\ unconstr$ is nonempty and for all $(r_a, r_b) \in OP\ T\ unconstr$, $r_a > A_2$ and $r_b \leq A_1$.
Lemma 3.3. Suppose Lemma 3.2 holds for values p, A_1 and A_2 , then, for this same choice of p , $OP\ T\ core$ is nonempty and there exist A_3 and A_4 with $A_3 > 0$ such that for all $(\theta_1, \theta_2) \in OP\ T\ core$, $\theta_1 > A_3$ and $\theta_2 < A_4$.

We will now explicitly construct a family of instances with candidate feature vectors parameterized by a value $\varepsilon \in R$ such that for sufficiently small $\varepsilon > 0$, the output of f fails the two axioms. Fix p, A_3 and A_4 from Lemma 3.3, and choose δ with $0 < \delta < 1$ such that $\delta A_4 - A_3 < 0$ ($\delta < A_3/A_4$ works if $A_4 > 0$, and otherwise, any $0 < \delta < 1$ will do). Each instance will have six candidates, which we will think of as two groups of three, $C = C_{core} \cup C_{copies}$. The first group $C_{core} = \{a, b, c\}$ will be the same as the three-candidate instance from above, while the second group $C_{copies} = \{a', b', c'\}$ will be new. The candidates a, b, c will still be located at $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. The candidates a', b', c' will be located near their undecorated counterparts at $x_{a'} := x_a + (-\varepsilon, 0)$, $x_{b'} := x_b + (-\varepsilon, 0)$ and $x_{c'} := x_c + (-\varepsilon, \delta \cdot \varepsilon)$.
Next, we describe the voter preferences. A p -fraction of voters will have the ranking $a > a' > b > b' > c' > c$, and the remaining $(1-p)$ -fraction of voters will have ranking $c' > c > b' > b > a' > a$. As long as $0 < \varepsilon < 1$ (which will be the case for our final chosen ε), these are both feasible rankings. The former is induced by the nondegenerate feature vector $(1, 1)/7$ and the latter by $(-1, 0)/8$.
For each $\varepsilon \in R$, let

$$L_\varepsilon(\theta) = \sum_{x \succ y} \ell(r_\theta(y) - r_\theta(x))$$

with $x \succ y \in \{0, 1 - p, p, 1\}$ be the loss function we are optimizing using candidate locations parameterized by ε .

We will show that for sufficiently small $\varepsilon > 0$, $\inf_{\theta \in R_{c'}} L_\varepsilon(\theta) > \inf_{\theta} L_\varepsilon(\theta)$. This means that f must output a ranking with $c > c'$. Observe that this is a PO violation because all voters agree that $c' > c$. Furthermore, this is a PMC violation because a majority of voters have the ranking $a > a' > b > b' > c' > c$, yet this is not the output.
Let $OP\ T(\varepsilon)$ be the set of vectors optimizing L_ε . The rest of the proof will follow from the following two lemmas, whose proofs are in Appendix A.

Lemma 3.4. $OP\ T(0) \subseteq R_{c'} > c$.

Lemma 3.5. Suppose $OP\ T(0) \subseteq R_{c'} > c$, then, for sufficiently small $\varepsilon > 0$, $\inf_{\theta \in R_{c'}} L_\varepsilon(\theta) > \inf_{\theta} L_\varepsilon(\theta)$.

- 7 This induces rewards $3, 3 - \varepsilon, 2, 2 - \varepsilon, (1 - \delta) \cdot \varepsilon$ and 0 for $a, a', b, b', c',$ and c , respectively.
- 8 This induces rewards $\varepsilon, 0, -1 + \varepsilon, -1, -2 + \varepsilon$ and -2 for $c', c, b', b, a',$ and a , respectively.

6

[form-feed in source extraction]
Finally, we handle the case that exists $x > 0$ such that $\ell(x) < \ell(0)$. Note that by convexity, this implies that for all $y < 0$, $\ell(y) > \ell(0) > \ell(x)$, so $\inf_{y \leq 0} \ell(y) < \inf_y \ell(y)$. Now, consider an instance with two candidates $\{a, b\}$ located at $x_a = (1, 0)$ and $x_b = (0, 1)$, and a single voter ranking $a > b$ (feasible via the parameter vector $(1, 0)$). It is possible to achieve a loss of $\ell(x)$, e.g., by outputting the parameter vector $(0, x)$. On the other hand, any θ inducing the ranking $a > b$ will be lower bounded by $\ell(0) > \ell(x)$ from above. Hence, f must output $b > a$, which is both a PO and PMC violation.

Expanded PaperInterface specification

```

∀ (loss : ℝ → ℝ),
  (∀ (input : ℝ), 0 ≤ loss input) →
    Monotone loss ∧ ConvexOn ℝ Set.univ loss ∧ StrictConvexOn ℝ Set.univ loss →
      snf (Set.range loss) < loss 0 →
        (∃ (positiveInput : ℝ),
          0 < positiveInput ∧
            loss positiveInput < loss 0 ∧
              ∀
                (rule :
                  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Unit 0
                    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
                      GetEtAI2024AlignmentAxioms.positiveInputFeatures)),
                  GetEtAI2024AlignmentAxioms.IsStandardLossMinimizing GetEtAI2024AlignmentAxioms.positiveInputFeatures
                    loss rule →
                      ¬AppliedModelingLib.Alignment.Axioms.ParetoOptimal
                        (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
                          GetEtAI2024AlignmentAxioms.positiveInputFeatures)
                    rule ∧
                      ¬AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
                        (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
                          GetEtAI2024AlignmentAxioms.positiveInputFeatures)
                    rule) v
                ∃ (weight : ℚ),
                  1 / 2 < weight ∧
                    weight < 1 ∧
                      ∃ (δ : ℝ),
                        0 < δ ∧
                          δ < 1 ∧
                            ∃ (majorityCount : ℕ),
                              ∃ minorityCount < majorityCount,
                                0 < majorityCount + minorityCount ∧
                                  ∃ (ε : ℝ),
                                    0 < ε ∧
                                      ε < 1 ∧
                                        ∀
                                          (rule :
                                            AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule

```

```

(Fin (majorityCount + minorityCount)) 4
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
 (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))),
GeEtAl2024AlignmentAxioms.IsStandardLossMinimizing
 (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ) loss rule  $\rightarrow$ 
 $\neg$ AppliedModelingLib.Alignment.Axioms.ParetoOptimal
 (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
 (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))
rule  $\wedge$ 
 $\neg$ AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
 (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
 (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))
rule

```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.ParetoOptimal](#)
- [GeEtAl2024AlignmentAxioms.IsStandardLossMinimizing](#)
- [GeEtAl2024AlignmentAxioms.positiveInputFeatures](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures](#)

Lean-elaborated claim roles

```

1. parameter:  $\mathbb{R} \rightarrow \mathbb{R}$ 
2. assumption:  $\forall (\text{input} : \mathbb{R}), 0 \leq \text{loss input}$ 
3. assumption: Monotone loss  $\wedge$  ConvexOn  $\mathbb{R}$  Set.univ loss v StrictConvexOn  $\mathbb{R}$  Set.univ loss
4. assumption:  $\text{slnf} (\text{Set.range loss}) < \text{loss } 0$ 
5. conclusion:  $(\exists (\text{positiveInput} : \mathbb{R}),$ 
   $0 < \text{positiveInput} \wedge$ 
   $\text{loss positiveInput} < \text{loss } 0 \wedge$ 
   $\forall$ 
    (rule :
      AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Unit 0
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
       GeEtAl2024AlignmentAxioms.positiveInputFeatures)),
      GeEtAl2024AlignmentAxioms.IsStandardLossMinimizing GeEtAl2024AlignmentAxioms.positiveInputFeatures loss rule  $\rightarrow$ 
       $\neg$ AppliedModelingLib.Alignment.Axioms.ParetoOptimal
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
       GeEtAl2024AlignmentAxioms.positiveInputFeatures)
      rule  $\wedge$ 
       $\neg$ AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
      (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
       GeEtAl2024AlignmentAxioms.positiveInputFeatures)
      rule) v
 $\exists (\text{weight} : \mathbb{Q}),$ 
 $1 / 2 < \text{weight} \wedge$ 
 $\text{weight} < 1 \wedge$ 
 $\exists (\delta : \mathbb{R}),$ 
 $0 < \delta \wedge$ 
 $\delta < 1 \wedge$ 
 $\exists (\text{majorityCount} : \mathbb{N}),$ 
 $\exists \text{minorityCount} < \text{majorityCount},$ 
 $0 < \text{majorityCount} + \text{minorityCount} \wedge$ 
 $\exists (\epsilon : \mathbb{R}),$ 
 $0 < \epsilon \wedge$ 
 $\epsilon < 1 \wedge$ 
 $\forall$ 
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule
    (Fin (majorityCount + minorityCount)) 4
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
     (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))),
    GeEtAl2024AlignmentAxioms.IsStandardLossMinimizing
    (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ) loss rule  $\rightarrow$ 
     $\neg$ AppliedModelingLib.Alignment.Axioms.ParetoOptimal
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
     (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))
    rule  $\wedge$ 
     $\neg$ AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking
     (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures  $\epsilon$   $\delta$ ))
    rule)

```

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theorem3_1_lossBasedImpossibility

Recorded source-to-Spec screening Verdict: matches

Reason: The target universally quantifies the source loss, retains nonnegativity from the standard-loss setup, the stated monotone-and-convex or strictly-convex alternative, and $\inf \text{loss} < \text{loss}(0)$. Its two conclusion branches are the positive-input two-candidate endpoint and the negative-input six-candidate construction from the proof; each says every ranking-level standard-loss minimizer violates both PO and PMC. The displayed standard-loss and restricted/global-infimum definitions implement the source sum over strict voter comparisons and Equation (1) without a changed optimization domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 318-319

Verbatim source input

Lemma 3.2. There exists a rational $p \in (1/2, 1]$ and values $A_1 < A_2$ with $A_2 > 0$ such that $OP\ T\ unconstr$ is nonempty and for all $(ra, rb) \in OP\ T\ unconstr$, $ra > A_2$ and $rb \leq A_1$.

[Next verbatim source excerpt]

We begin with a small instance of just three candidates $C\ core = \{a, b, c\}$ to gain some traction on how ℓ behaves. We will later extend this instance with additional candidates to demonstrate a profile where PO and PMC fail. The candidates will have feature vectors $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. Furthermore, a p -fraction of voters (for p to be chosen later) will rank $a > b > c$, while the remaining $(1 - p)$ -fraction will have inverted preferences, ranking $c > b > a$.
Let

$$Lcore(\theta) := \sum_{x \in C\ core} w_x \ell(r\theta(x) - r\theta(x))$$

Typically, BTL is presented as choosing θ maximizing the likelihood of seeing the pairwise comparisons we observed, assuming that $\Pr[a > b] = \frac{e^{r\theta(a)}}{e^{r\theta(a)} + e^{r\theta(b)}}$. That is, we choose θ maximizing

$$\sum_{(a,b) \in \mathcal{Q}} \log \frac{e^{r\theta(a)}}{e^{r\theta(a)} + e^{r\theta(b)}}$$

By taking the log and swapping the sign, we see that this is equivalent to minimizing $\sum_{(a,b) \in \mathcal{Q}} \log(1 + e^{r\theta(b) - r\theta(a)})$.

It so happens that these rankings are feasible, but for now we will not worry about this as loss function-based rules still make sense regardless of whether the inputs are feasible. For the final example, we will ensure that the rankings are feasible.

5

[form-feed in source extraction]

with $w_x \geq 0$ and $\sum_x w_x = 1$. Let $g(x) = p \cdot \ell(-x) + (1 - p) \ell(x)$. Note that we can rewrite $Lcore$ as

$$Lcore(\theta) = g(r\theta(a) - r\theta(b)) + g(r\theta(a) - r\theta(c)) + g(r\theta(b) - r\theta(c)).$$

Note that $r\theta(c) = 0$ for all θ , so we can simplify this to

$$Lcore(\theta) = g(r\theta(a) - r\theta(b)) + g(r\theta(a)) + g(r\theta(b)).$$

We will consider an unconstrained version of this problem where we are free to choose rewards $ra, rb \in \mathbb{R}$ arbitrarily, and later show by which vectors θ these optimal values can be induced. That is, we will first find $ra, rb \in \mathbb{R}$ minimizing

$$Lunconstr(ra, rb) := g(ra - rb) + g(ra) + g(rb).$$

Let $OP\ T\ core = \{\theta \mid Lcore(\theta) = \inf_{\theta'} Lcore(\theta')\}$ and $OP\ T\ unconstr = \{(ra, rb) \mid Lunconstr(ra, rb) = \inf_{ra', rb'} Lunconstr(ra', rb')\}$ be the set of minimizers for these two loss functions. In Appendix A, we establish the following results about these optimal sets.

Lemma 3.2. There exists a rational $p \in (1/2, 1]$ and values $A_1 < A_2$ with $A_2 > 0$ such that $OP\ T\ unconstr$ is nonempty and for all $(ra, rb) \in OP\ T\ unconstr$, $ra > A_2$ and $rb \leq A_1$.

[Next verbatim source excerpt]

A.1 Proof of Lemma 3.2

Proof. We begin with some observations on g . First, we have that since ℓ is nonnegative, g must also be nonnegative. This along with the fact that $\lim_{x \rightarrow \infty} \ell(x) = \infty$, we have that both $\lim_{x \rightarrow \infty} g(x) = \infty$ and $\lim_{x \rightarrow -\infty} g(x) = \infty$. Together, these imply that $Lunconstr$ attains a minimum. Indeed, $Lunconstr(0, 0) = 3g(0)$, and there is some bound B such that for all $x > B$ and $x < -B$, $g(x) > 3g(0)$. We can therefore restrict the optimization problem to $ra, rb \in [-B, B]$ without changing the solutions. Since $Lunconstr$ is continuous and $[-B, B]^2$ is compact, a minimum is attained.

Next, note that g is convex because compositions of convex functions with monotonic functions and convex combinations of convex functions are convex [24]. From this, we claim that if there is an optimal solution (ra, rb) , then $(ra, ra/2)$ is also an optimal solution. Indeed, fix such an (ra, rb) ,

$$\begin{aligned} Lunconstr(ra, ra/2) &= g(ra - ra/2) + g(ra) + g(ra/2) \\ &= g(ra/2) + 2g(ra/2) \\ &= g(ra) + 2g(1/2(ra - rb) + 1/2rb) \\ &\leq g(ra) + 2(1/2g(ra - rb) + 1/2g(rb)) \\ &= Lunconstr(ra, rb), \end{aligned}$$

where the inequality comes from convexity. This implies that $(ra, ra/2)$ is also optimal.

By above, we have that if (rb, ra) is optimal, it must be the case that ra minimizes

$$h(ra) := 2g(ra/2) + g(ra).$$

Observe that h is again convex by monotonic composition and convex combinations.

Next, we will make use of the following facts about convex functions. Although they need not be differentiable, right- and left-hand derivatives always exist. For a function k these are defined as

$$\begin{aligned} k_+(x) &= \lim_{h \rightarrow 0^+} \frac{k(x+h) - k(x)}{h} \\ k_-(x) &= \lim_{h \rightarrow 0^-} \frac{k(x+h) - k(x)}{h} \end{aligned}$$

Further, if k is convex, we have that [24]:

- (i) k_+ and k_- are nondecreasing,
- (ii) $k_-(x) \leq k_+(x)$ for every x ,
- (iii) x minimizes k if and only if $k_-(x) \leq 0 \leq k_+(x)$,
- (iv) k_+ and k_- are right and left continuous, respectively.

They also follow standard linearity and chain rule properties, which allow for simpler computation. For example:

$$\text{if } k(x) = \alpha\alpha(x) + b\beta(x), \text{ then } k^+(x) = \alpha\alpha^+(x) + b\beta^+(x),$$

if $k(x) = \alpha(\gamma x)$, then $k^+(x) = \gamma\alpha^+(x)$ if $\gamma \geq 0$, and $k^+(x) = \gamma\alpha^-(x)$ if $\gamma < 0$, (all of these hold for left-hand derivatives by swapping the positions of $+$ and $-$) [9]. Next, we claim that we can find a valid p (rational with $1/2 < p < 1$) and $w > 0$ such that

$g^+(w) > 0$, while $h^+(w) < 0$. To that end, expanding the first derivative, we have

$$g^+(x) = -p\ell'^-(-x) + (1-p)\ell'^+(x).$$

As long as $\ell'^-(-x) + \ell'^+(x) > 0$, this is strictly more than 0 for p satisfying

$$p < \frac{\ell'^-(-x) + \ell'^+(x)}{\ell'^-(-x) + \ell'^+(x)} \quad (2)$$

19

[form-feed in source extraction]

For h ,

$$h^+(x) = 2 \cdot (1/2)g^+(x/2) + g^+(x)$$

$$\begin{aligned} &= -p\ell'^-(-x/2) + (1-p)\ell'^+(x/2) - p\ell'^-(-x) + (1-p)\ell'^+(x). \\ &= -p[\ell'^-(-x/2) + \ell'^-(-x)] + (1-p)[\ell'^+(x/2) + \ell'^+(x)]. \end{aligned}$$

As long as $\ell'^-(-x/2) + \ell'^-(-x) + \ell'^+(x/2) + \ell'^+(x) > 0$, then this is strictly less than 0 for

$$p > \frac{\ell'^+(x/2) + \ell'^+(x)}{\ell'^-(-x/2) + \ell'^-(-x) + \ell'^+(x/2) + \ell'^+(x)} \quad (3)$$

Now, choose $w > 0$ such that $\ell'^-(-w/2) > \ell'^-(-w) \geq 0$. This is possible by using the following procedure. We know $\ell'^-(0) > 0$ (as otherwise $\ell(0) < \ell(x)$ for all $x < 0$, contradicting our assumption on ℓ). We will split into cases depending on whether ℓ is nondecreasing or strictly-convex (at least one must be true by the theorem assumptions).

First, suppose ℓ is non-decreasing. This implies that $\ell'^-(x) \geq 0$ for all x . We know that there is a point $x < 0$ such that $\ell'^-(x) < \ell'^-(0)$ (as otherwise ℓ would eventually become negative). Let $d = \ell'^-(x)$. Take w such that

$$-w = \sup\{x \mid \ell'^-(x) = d\}.$$

Note that $\ell'^-(-w) = d$ because ℓ'^- is left continuous (from (iv) above). Since $-w < 0$, so $-w/2 > -w$. This implies that $\ell'^-(-w/2) > d$, as otherwise $-w$ would not be the supremum of such points. In addition, we have that $d \geq 0$ because ℓ is nondecreasing.

Next, suppose ℓ is strictly convex. then ℓ'^- is strictly increasing. Further, since it is left continuous and $\ell'(0) > 0$, there is a $\gamma > 0$ such that for all $x \in [-\gamma, 0]$, $\ell'^-(x) \geq 0$. Therefore, choosing $w = \gamma$ will do: $\ell'^-(-\gamma) \geq 0$ by choice of γ , and $-\gamma/2 > -\gamma$, so $\ell'^-(-\gamma/2) > \ell'^-(-\gamma)$ since ℓ'^- is strictly increasing.

For this choice of w , we first claim that the preconditions of denominators being positive hold for (2) and (3). Indeed, let us write z_1, z_2, z_3, z_4 for $\ell'^-(-2w), \ell'^-(-w), \ell'^+(w), \ell'^+(2w)$. We know that $z_1 \leq z_2 \leq z_3 \leq z_4$

by properties of convexity, and since $\ell'^-(-w/2) > \ell'^-(-w) \geq 0$, we have that $0 \leq z_1 < z_2$. The denominators are of the form $z_1 + z_4$ and $z_1 + z_2 + z_3 + z_4$, which are now both necessarily positive. In addition, we also claim that a rational p with $1/2 < p < 1$ satisfying both inequalities (2) and (3) will exist. Note that inequality (2) can now be represented as $z_1 z_4$

$$\frac{z_4}{z_1 + z_4}, \text{ and we have that}$$

$$\frac{z_4}{z_1 + z_4} \leq 1$$

because $0 \leq z_1 < z_4$. Additionally, inequality (3) can be represented as

$$\frac{z_3 + z_4}{z_1 + z_2 + z_3 + z_4} < \frac{z_4}{z_1 + z_4}$$

which is at least $1/2$ because $z_3 + z_4 > z_1 + z_2$. Finally, we have

$$\frac{z_2 + z_3}{z_1 + z_2 + z_3 + z_4} < \frac{z_4}{z_1 + z_4}$$

which implies that

$$\frac{z_3 + z_4}{z_1 + z_2 + z_3 + z_4} < \frac{z_4}{z_1 + z_4}$$

Hence, there exists some rational p in this interval, which is necessarily between $1/2$ and 1 , as needed.

To summarize, we have found a valid p and value $w > 0$ such that $h^+(w) < 0$ while $g^+(w) > 0$.

Because h^+ is right continuous (from (iv) above), there is some $\gamma > 0$ such that $h^+(w + \gamma) < 0$ as well. We will now show for all $(r_a, r_b) \in \text{OPT unconstr}$, $r_a > w + \gamma$ and $r_b \leq w$. Indeed, note that r_a must minimize h , so $h^+(r_a) \geq 0$ (from (iii) above), which implies $r_a > w + c$. For r_b , suppose

20

[form-feed in source extraction]

for a contradiction $r_b > w$ as well. Note that $g^+(w) > 0$ means g is increasing to the right of w . Let

$d = \min(r_a, r_b) - w > 0$. Consider $(r_a, r_b) = (r_a - d, r_b - d)$. We then have

$$\begin{aligned} \text{Lunconstr}(r_a', r_b') &= g(r_a' - r_b') + g(r_a') + g(r_b') \\ &= g(r_a - r_b) + g(r_a') + g(r_b') \\ &< g(r_a - r_b) + g(r_a) + g(r_b) \\ &= \text{Lunconstr}(r_a, r_b). \end{aligned}$$

where the equality holds because $r_a' - r_b' = r_a - r_b$ and the inequality because g is increasing to the

right of w . Therefore, we reach a contradiction, because then (r_a, r_b) would not be optimal. Thus, we have found values satisfying the lemma statement with $A1 = w$ and $A2 = w + \gamma$.

Expanded PaperInterface specification

```

∀ {loss : ℝ → ℝ} {negativeInput : ℝ},
  (∀ (input : ℝ), 0 ≤ loss input) →
  Monotone loss ∧ ConvexOn ℝ Set.univ loss v StrictConvexOn ℝ Set.univ loss →
  negativeInput < 0 →
  loss negativeInput < loss 0 →
  ∃ (p : ℚ),
    1 / 2 < ↑p ∧
    ↑p ≤ 1 ∧
    ∃ (A1 : ℝ) (A2 : ℝ),
      A1 < A2 ∧
      0 < A2 ∧
      ∃ (rewards : ℝ × ℝ),
        GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
          (GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss
            (GeEtAl2024AlignmentAxioms.weightedLoss (↑p) loss))
          rewards ∧
      ∀ (rewards : ℝ × ℝ),
        GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
          (GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss
            (GeEtAl2024AlignmentAxioms.weightedLoss (↑p) loss))
          rewards →
      rewards.1 > A2 ∧ rewards.2 ≤ A1

```

Retained governing declarations

- [GeEtAl2024AlignmentAxioms.IsGlobalMinimizer](#)
- [GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss](#)
- [GeEtAl2024AlignmentAxioms.weightedLoss](#)

Lean-elaborated claim roles

```

1. parameter: ℝ → ℝ
2. parameter: ℝ
3. assumption: ∀ (input : ℝ), 0 ≤ loss input
4. assumption: Monotone loss ∧ ConvexOn ℝ Set.univ loss v StrictConvexOn ℝ Set.univ loss
5. assumption: negativeInput < 0
6. assumption: loss negativeInput < loss 0
7. conclusion: ∃ (p : ℚ),
  1 / 2 < ↑p ∧
  ↑p ≤ 1 ∧
  ∃ (A1 : ℝ) (A2 : ℝ),
    A1 < A2 ∧
    0 < A2 ∧
    ∃ (rewards : ℝ × ℝ),
      GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
        (GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss (GeEtAl2024AlignmentAxioms.weightedLoss (↑p) loss))
        rewards ∧
      ∀ (rewards : ℝ × ℝ),
        GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
          (GeEtAl2024AlignmentAxioms.unconstrainedCoreLoss
            (GeEtAl2024AlignmentAxioms.weightedLoss (↑p) loss))
          rewards →
      rewards.1 > A2 ∧ rewards.2 ≤ A1

```

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.lemma3_2_unconstrainedCoreSeparation

Recorded source-to-Spec screening Verdict: matches

Reason: The source Lemma 3.2 and its displayed Lunconstr context match the expanded binders and conclusion: a rational p in $(1/2, 1]$, $A1 < A2$ with $A2 > 0$, a nonempty global-minimizer set, and the universal $r_a > A2$, $r_b \leq A1$ bounds. The material definitions expand g and Lunconstr exactly to $p \cdot l(-x) + (1-p) \cdot l(x)$ and $g(r_a - r_b) + g(r_a) + g(r_b)$; the loss hypotheses shown in the target are the source theorem context used by the lemma proof.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 320-322

Verbatim source input

Lemma 3.3. Suppose Lemma 3.2 holds for values p, A_1 and A_2 , then, for this same choice of p , $OP\ T\ core$ is nonempty and there exist A_3 and A_4 with $A_3 > 0$ such that for all $(\theta_1, \theta_2) \in OP\ T\ core$, $\theta_1 > A_3$ and $\theta_2 < A_4$.

[Next verbatim source excerpt]

We begin with a small instance of just three candidates $C\ core = \{a, b, c\}$ to gain some traction on how ℓ behaves. We will later extend this instance with additional candidates to demonstrate a profile where PO and PMC fail. The candidates will have feature vectors $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. Furthermore, a p -fraction of voters (for p to be chosen later) will rank $a > b > c$, while the remaining $(1 - p)$ -fraction will have inverted preferences, ranking $c > b > a$.⁵

$$Lcore(\theta) := \sum_{x \succ y \in C\ core} w_{x \succ y} \ell(r_\theta(y) - r_\theta(x))$$

Typically, BTL is presented as choosing θ maximizing the likelihood of seeing the pairwise comparisons we observed, assuming that $\Pr[a > b] = \frac{e^{r_\theta(a)}}{e^{r_\theta(a)} + e^{r_\theta(b)}}$. That is, we choose θ maximizing $\sum_{a > b} \log \frac{e^{r_\theta(a)}}{e^{r_\theta(a)} + e^{r_\theta(b)}}$.

By taking the log and swapping the sign, we see that this is equivalent to minimizing $\sum_{a > b} \log(1 + e^{r_\theta(b) - r_\theta(a)})$.

It so happens that these rankings are feasible, but for now we will not worry about this as loss function-based rules still make sense regardless of whether the inputs are feasible. For the final example, we will ensure that the rankings are feasible.

5

[form-feed in source extraction]

with $w_{x \succ y} \in \{1 - p, p\}$ be the loss function on this instance (scaling $n_{x \succ y}$ down to $w_{x \succ y}$ leads to an equivalent formulation). Let $g(x) = p \cdot \ell(-x) + (1 - p)\ell(x)$. Note that we can rewrite L_{com} as $Lcore(\theta) = g(r_\theta(a) - r_\theta(b)) + g(r_\theta(a) - r_\theta(c)) + g(r_\theta(b) - r_\theta(c))$.

Note that $r_\theta(c) = 0$ for all θ , so we can simplify this to

$$Lcore(\theta) = g(r_\theta(a) - r_\theta(b)) + g(r_\theta(a)) + g(r_\theta(b)).$$

We will consider an unconstrained version of this problem where we are free to choose rewards $r_a, r_b \in \mathbb{R}$ arbitrarily, and later show by which vectors θ these optimal values can be induced. That is, we will first find $r_a, r_b \in \mathbb{R}$ minimizing

$$Lunconstr(r_a, r_b) := g(r_a - r_b) + g(r_a) + g(r_b).$$

Let $OP\ T\ core = \{\theta \mid Lcore(\theta) = \inf_{\theta'} Lcore(\theta')\}$ and $OP\ T\ unconstr = \{(r_a, r_b) \mid Lunconstr(r_a, r_b) = \inf_{r_a', r_b'} Lunconstr(r_a', r_b')\}$ be the set of minimizers for these two loss functions. In Appendix A, we establish the following results about these optimal sets.

Lemma 3.2. There exists a rational $p \in (1/2, 1]$ and values $A_1 < A_2$ with $A_2 > 0$ such that

$OP\ T\ unconstr$ is nonempty and for all $(r_a, r_b) \in OP\ T\ unconstr$, $r_a > A_2$ and $r_b \leq A_1$.

Lemma 3.3. Suppose Lemma 3.2 holds for values p, A_1 and A_2 , then, for this same choice of p , $OP\ T\ core$ is nonempty and there exist A_3 and A_4 with $A_3 > 0$ such that for all $(\theta_1, \theta_2) \in OP\ T\ core$, $\theta_1 > A_3$ and $\theta_2 < A_4$.

[Next verbatim source excerpt]

A.2 Proof of Lemma 3.3

Proof. Fix p inducing a nonempty $OP\ T\ unconstr$ satisfying Lemma 3.2 with values A_1 and A_2 .

We first claim that for any (r_a, r_b) (regardless of optimality), it is possible to find a θ such that $r_\theta(a) = r_a$ and $r_\theta(b) = r_b$. Indeed, note that

$$[[2, 1], [1, 1]] \cdot [\theta_1, \theta_2] = [r_\theta(a), r_\theta(b)].$$

Since $M = [[2, 1],$

$[1, 1]]$ is invertible with inverse

$$M^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix},$$

for any r_a, r_b , we can simply set

$$\theta = M^{-1} [r_a, r_b].$$

Thus, $OP\ T\ com$ is nonempty, and is simply the image of $OP\ T\ unconstr$ under M^{-1} . Now, fix

$(r_a, r_b) \in OP\ T\ unconstr$,

and let $\theta = M^{-1} [r_a, r_b]$.

By assumption, we have $r_a > A_2$ and $r_b \leq A_1$.

This implies that $\theta_1 = r_a - r_b > A_2 - A_1$, while $\theta_2 = 2r_b - r_a < 2A_1 - A_2$. Thus, setting $A_3 = A_2 - A_1$ and $A_4 = 2A_1 - A_2$ satisfy the desired properties.

Expanded PaperInterface specification

$\forall \{g : \mathbb{R} \rightarrow \mathbb{R}\} \{A_1 A_2 : \mathbb{R}\},$

```

A1 < A2 →
(∃ (rewards : ℝ × ℝ),
  GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.unconstrainedCoreLoss g) rewards) →
(∀ (rewards : ℝ × ℝ),
  GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.unconstrainedCoreLoss g) rewards →
  rewards.1 > A2 ∧ rewards.2 ≤ A1) →
∃ (A3 : ℝ) (A4 : ℝ),
  0 < A3 ∧
  ∃ (parameter : ℝ × ℝ),
    GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.coreLoss g) parameter ∧
    ∀ (parameter : ℝ × ℝ),
      GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.coreLoss g) parameter →
        parameter.1 > A3 ∧ parameter.2 < A4

```

Retained governing declarations

- [GeEtAI2024AlignmentAxioms.IsGlobalMinimizer](#)
- [GeEtAI2024AlignmentAxioms.coreLoss](#)
- [GeEtAI2024AlignmentAxioms.unconstrainedCoreLoss](#)

Lean-elaborated claim roles

```

1. parameter: ℝ → ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: A1 < A2
5. assumption: ∃ (rewards : ℝ × ℝ),
  GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.unconstrainedCoreLoss g) rewards
6. assumption: ∀ (rewards : ℝ × ℝ),
  GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.unconstrainedCoreLoss g) rewards →
  rewards.1 > A2 ∧ rewards.2 ≤ A1
7. conclusion: ∃ (A3 : ℝ) (A4 : ℝ),
  0 < A3 ∧
  ∃ (parameter : ℝ × ℝ),
    GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.coreLoss g) parameter ∧
    ∀ (parameter : ℝ × ℝ),
      GeEtAI2024AlignmentAxioms.IsGlobalMinimizer (GeEtAI2024AlignmentAxioms.coreLoss g) parameter →
        parameter.1 > A3 ∧ parameter.2 < A4

```

Lean proof endpoint (built separately)

GeEtAI2024AlignmentAxioms.lemma3_3_coreMinimizers

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded premises are precisely the Lemma 3.2 premises for fixed g , $A1$, and $A2$, including nonempty unconstrained optima and their two coordinate bounds. Its conclusion gives nonempty core optima and $A3 > 0$ with the stated $\theta_1 > A3$ and $\theta_2 < A4$ bounds. The displayed core-reward map is $(2\theta_1 + \theta_2, \theta_1 + \theta_2)$, exactly the reward substitution in the source proof.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 351

Verbatim source input

Lemma 3.4. $OP T(0) \subseteq Rc' > c$.

[Next verbatim source excerpt]

We will now explicitly construct a family of instances with candidate feature vectors parameterized by a value $\epsilon \in \mathbb{R}$ such that for sufficiently small $\epsilon > 0$, the output of f fails the two axioms. Fix $p, A3$ and $A4$ from Lemma 3.3, and choose δ with $0 < \delta < 1$ such that $\delta A4 - A3 < 0$ ($\delta < A3/A4$ works if $A4 > 0$, and otherwise, any $0 < \delta < 1$ will do). Each instance will have six candidates, which we will think of as two groups of three, $C = C_{core} \cup C_{copies}$. The first group $C_{core} = \{a, b, c\}$ will be the same as the three-candidate instance from above, while the second group $C_{copies} = \{a', b', c'\}$ will be new. The candidates a, b, c will still be located at $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. The candidates a', b', c' will be located near their undecorated counterparts at $x_{a'} := x_a + (-\epsilon, 0)$, $x_{b'} := x_b + (-\epsilon, 0)$ and $x_{c'} := x_c + (-\epsilon, \delta \cdot \epsilon)$. Next, we describe the voter preferences. A p -fraction of voters will have the ranking $a > a' > b > b' > c' > c$, and the remaining $(1-p)$ -fraction of voters will have ranking $c' > c > b' > b > a' > a$. As long as $0 < \epsilon < 1$ (which will be the case for our final chosen ϵ), these are both feasible rankings. The former is induced by the nondegenerate feature vector $(1, 1)^T$ and the latter by $(-1, 0)^T$. For each $\epsilon \in \mathbb{R}$, let

$$L_\epsilon(\theta) = \sum_{x \succ y \in C} wx \succ y \text{ ell}(r_\theta(y) - r_\theta(x))$$

with $wx \succ y \in \{0, 1 - p, p, 1\}$ be the loss function we are optimizing using candidate locations parameterized by ϵ .

We will show that for sufficiently small $\epsilon > 0$, $\inf_{\theta \in Rc'} L_\epsilon(\theta) > \inf_{\theta \in C} L_\epsilon(\theta)$. This means that f must output a ranking with $c > c'$. Observe that this is a PO violation because all voters agree that $c' > c$. Furthermore, this is a PMC violation because a majority of voters have the ranking $a > a' > b > b' > c' > c$, yet this is not the output. Let $OPT(\epsilon)$ be the set of vectors optimizing L_ϵ . The rest of the proof will follow from the following two lemmas, whose proofs are in Appendix A.

Lemma 3.4. $OPT(0) \subseteq Rc' > c$.

Lemma 3.5. Suppose $OPT(0) \subseteq Rc' > c$, then, for sufficiently small $\epsilon > 0$, $\inf_{\theta \in Rc'} L_\epsilon(\theta) >$

[Next verbatim source excerpt]

A.3 Proof of Lemma 3.4

Proof. We first claim that $OPT(0) = OPT_{core}$. This will follow from showing

$$L_0(\theta) = 4 \cdot L_{core}(\theta) + 3 \cdot \text{ell}(0).$$

Indeed, in L_0 , the copied candidates are in exactly the same location as their counterparts. Hence, each term in L_{core} appears 4 times, one for each combination of original and copy. In addition to these, there are the 6 terms for each ordered pair of (a, a') , (b, b') , and (c, c') . Note that, each $r_\theta(x) = r_\theta(x')$ for each $x \in \{a, b, c\}$ regardless of θ since they are in the same location. Therefore, the ell portion is always $\text{ell}(0)$, and the corresponding $wx \succ x'$ and $wx' \succ x$ terms add up to one for each pair. Hence, the total sum of these terms is $3 \cdot \text{ell}(0)$. Since L_0 is equivalent to L_{core} up to positive scaling and translation, they have the same optima.

Finally, fix a $\theta \in OPT(0)$. Since $\theta \in OPT_{core}$, by Lemma 3.3, $\theta_1 > A3$ and $\theta_2 < A4$. Thus,

$$r_\theta(c') = \epsilon \cdot (-\theta_1 + \delta \theta_2) < \epsilon(-A3 + \delta A4) < 0 = r_\theta(c)$$

by choice of δ . Hence, $\theta \in Rc' > c$

Expanded PaperInterface specification

```
∀ (δ weight : ℝ) (loss : ℝ → ℝ) (parameter : ℝ × ℝ),
  GeEtAl2024AlignmentAxioms.IsGlobalMinimizer (GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective 0 δ weight loss)
    parameter →
  AppliedModelingLib.Alignment.Axioms.linearReward (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
    (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures 0 δ) 5 ≤
  AppliedModelingLib.Alignment.Axioms.linearReward (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
    (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures 0 δ) 4
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.IsGlobalMinimizer](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective](#)
- [GeEtAl2024AlignmentAxioms.twoCoordinateParameter](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
4. parameter: $\mathbb{R} \times \mathbb{R}$

5. assumption: GeEtAl2024AlignmentAxioms.IsGlobalMinimizer (GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective 0 6 weight loss) parameter
6. conclusion: AppliedModelingLib.Alignment.Axioms.linearReward (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter) (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures 0 6) 5 ≤ AppliedModelingLib.Alignment.Axioms.linearReward (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter) (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures 0 6) 4

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.lemma3_4_literalZeroWeakInclusion

Recorded source-to-Spec screening Verdict: matches
Reason: Selected source item 'lemma3_4'. The source states $\text{OPT}(0) \subseteq R_{\{c' > c\}}$ and identifies the zero-perturbation copy locations with their originals. In the displayed Lean feature family, candidate 4 is c' with features $(-\varepsilon, \delta\varepsilon)$ and candidate 5 is c with features $(0, 0)$, so at $\varepsilon = 0$ the conclusion $\text{reward}(c) \leq \text{reward}(c')$ holds with equality for every parameter. The premise identifies the parameter as a global minimizer of the displayed $\varepsilon = 0$ six-candidate objective, hence exactly gives membership in $\text{OPT}(0)$; the broader quantification over δ , weight, and loss specializes to the source's chosen values without excluding any source-admissible case. The displayed weighted-loss and ranking definitions also preserve the source's two ranking types and pairwise-loss objective. Thus the source inclusion, its weak endpoint, quantifiers, and domain are preserved.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 5

Source locator: cited publication, lines 353-354

Verbatim source input

Lemma 3.5. Suppose $OP\ T\ (0) \subseteq Rc' > c$, then, for sufficiently small $\varepsilon > 0$, $\inf \theta: \theta \in Rc' > c\ L\varepsilon\ (\theta) > \inf \theta\ L\varepsilon\ (\theta)$.

[Next verbatim source excerpt]

We will now explicitly construct a family of instances with candidate feature vectors parameterized by a value $\varepsilon \in \mathbb{R}$ such that for sufficiently small $\varepsilon > 0$, the output of f fails the two axioms. Fix $p, A3$ and $A4$ from Lemma 3.3, and choose δ with $0 < \delta < 1$ such that $\delta A4 - A3 < 0$ ($\delta < A3/A4$ works if $A4 > 0$, and otherwise, any $0 < \delta < 1$ will do). Each instance will have six candidates, which we will think of as two groups of three, $C = C_{core} \cup C_{copies}$. The first group $C_{core} = \{a, b, c\}$ will be the same as the three-candidate instance from above, while the second group $C_{copies} = \{a', b', c'\}$ will be new. The candidates a, b, c will still be located at $x_a := (2, 1)$, $x_b := (1, 1)$, and $x_c := (0, 0)$, respectively. The candidates a', b', c' will be located near their undecorated counterparts at $x_{a'} := x_a + (-\varepsilon, 0)$, $x_{b'} := x_b + (-\varepsilon, 0)$ and $x_{c'} := x_c + (-\varepsilon, \delta \cdot \varepsilon)$. Next, we describe the voter preferences. A p -fraction of voters will have the ranking $a > a' > b > b' > c' > c$, and the remaining $(1-p)$ -fraction of voters will have ranking $c' > c > b' > b > a' > a$. As long as $0 < \varepsilon < 1$ (which will be the case for our final chosen ε), these are both feasible rankings. The former is induced by the nondegenerate feature vector $(1, 1)^T$ and the latter by $(-1, 0)^T$. For each $\varepsilon \in \mathbb{R}$, let

$$L\varepsilon\ (\theta) = \sum_{x \succ y \in C} wx > y \ell(r\theta(y) - r\theta(x))$$

with $wx > y \in \{0, 1 - p, p, 1\}$ be the loss function we are optimizing using candidate locations parameterized by ε .

We will show that for sufficiently small $\varepsilon > 0$, $\inf \theta \in Rc' > c\ L\varepsilon\ (\theta) > \inf \theta\ L\varepsilon\ (\theta)$. This means that f must output a ranking with $c > c'$. Observe that this is a PO violation because all voters agree that $c' > c$. Furthermore, this is a PMC violation because a majority of voters have the ranking $a > a' > b > b' > c' > c$, yet this is not the output. Let $OP\ T\ (\varepsilon)$ be the set of vectors optimizing $L\varepsilon$. The rest of the proof will follow from the following two lemmas, whose proofs are in Appendix A.

Lemma 3.4. $OP\ T\ (0) \subseteq Rc' > c$.

Lemma 3.5. Suppose $OP\ T\ (0) \subseteq Rc' > c$, then, for sufficiently small $\varepsilon > 0$, $\inf \theta: \theta \in Rc' > c\ L\varepsilon\ (\theta) > \inf \theta\ L\varepsilon\ (\theta)$.

[Next verbatim source excerpt]

A.4 Proof of Lemma 3.5

Proof. We first show that when optimizing each $L\varepsilon$, it is sufficient to consider only θ coming from a bounded region. Indeed, observe that $L\varepsilon\ (0) = 62 \ell(0)$ for all ε . Since $\lim_{x \rightarrow \infty} \ell(x) = \infty$, we

21

[form-feed in source extraction]

(6) $\ell(0) = \varepsilon$
can find some $B > 0$ such that for all $x > B$, $\ell(x) > 21 - p = L1 - p$ (0).
For a pair of candidates

$x' = y \in C_{com}$, in the two terms concerning these candidates, we have
 $wx > y \ell(r\theta(y) - r\theta(x)) + wy > x \ell(r\theta(x) - r\theta(y))$
 $\geq (1 - p) (\ell(r\theta(y) - r\theta(x)) + \ell(r\theta(x) - r\theta(y)))$
 $\geq (1 - p) \ell(|r\theta(y) - r\theta(x)|).$

Applying this to $\{a, b\}$ and $\{b, c\}$,

$$L\varepsilon\ (\theta) \geq (1 - p) (\ell(|r\theta(a) - r\theta(b)|) + \ell(|r\theta(b) - r\theta(c)|)) \\ = (1 - p) (\ell(|\theta_1|) + \ell(|\theta_1 + \theta_2|))$$

This implies that we may restrict our attention to θ in the region

$$R_{bounded} = \{\theta \mid |\theta_1| \leq B, |\theta_2| \leq 2B\}.$$

/ $R_{bounded}$, either $|\theta_1| > B$ or $|\theta_1 + \theta_2| > B$. In either case, we have $L\varepsilon\ (\theta) \geq$

Indeed, for $\theta \in$

$$(1 - p) \ell(B) > L\varepsilon\ (0).$$

Note that $L\varepsilon\ (\theta)$ is continuous not only in θ , but also in ε . Additionally, $R_{bounded}$ is closed and bounded, and hence, compact. Therefore, by Berge's Maximum Theorem, $OP\ T\ (\varepsilon)$ is nonempty and upper semi-continuous in ε [3]. As per the definition of upper semi-continuous, since $OP\ T\ (0) \subseteq$

$Rc' > c$, an open set, for sufficiently small $\varepsilon > 0$, $OP\ T\ (\varepsilon) \subseteq Rc' > c$. Finally, note that $Rc > c_n$. $R_{bounded}$ is compact, so a minimum is attained, and this minimum must therefore be strictly larger than the values attained by members of $OP\ T\ (\varepsilon)$.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Under the corrected strict minimizer cone, the literal six-candidate source objective has a strict infimum gap on the complete weak bad half-space for all $\hookrightarrow$ sufficiently small positive perturbations.

Archival source anchor: cited publication, lines 353-354

Expanded PaperInterface specification

$\forall \{loss : \mathbb{R} \rightarrow \mathbb{R}\} \{negativeInput\ \delta : \mathbb{R}\} \{weight : \mathbb{Q}\} \{base : \mathbb{R} \times \mathbb{R}\},$
 $1/2 < \uparrow weight \rightarrow$

```

↑weight < 1 →
(∀ (input : ℝ), 0 ≤ loss input) →
ConvexOn ℝ Set.univ loss →
negativeInput < 0 →
loss negativeInput < loss 0 →
GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
  (GeEtAl2024AlignmentAxioms.coreLoss (GeEtAl2024AlignmentAxioms.weightedLoss (↑weight) loss)) base →
(∀ (parameter : ℝ × ℝ),
  GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
    (GeEtAl2024AlignmentAxioms.coreLoss (GeEtAl2024AlignmentAxioms.weightedLoss (↑weight) loss))
      parameter →
    GeEtAl2024AlignmentAxioms.copyBelowOriginalCone δ parameter) →
∃ (bound : ℝ) (radius : ℝ),
  0 < bound ∧
  0 < radius ∧
  ∀ (ε : ℝ),
    0 < ε →
    ε < radius →
    sInf
      (GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective ε δ (↑weight) loss "
        {parameter : ℝ × ℝ |
          AppliedModelingLib.Alignment.Axioms.linearReward
            (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
            (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures ε δ) 5 ≤
            AppliedModelingLib.Alignment.Axioms.linearReward
              (GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
              (GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures ε δ) 4}) >
        sInf
          (Set.range (GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective ε δ (↑weight) loss))

```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.linearReward](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.IsGlobalMinimizer](#)
- [GeEtAl2024AlignmentAxioms.copyBelowOriginalCone](#)
- [GeEtAl2024AlignmentAxioms.coreLoss](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures](#)
- [GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective](#)
- [GeEtAl2024AlignmentAxioms.twoCoordinateParameter](#)
- [GeEtAl2024AlignmentAxioms.weightedLoss](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. parameter: $\mathbb{Q}$
5. parameter: $\mathbb{R} \times \mathbb{R}$
6. assumption: $1/2 < \uparrow \text{weight}$
7. assumption: $\uparrow \text{weight} < 1$
8. assumption: $\forall (\text{input} : \mathbb{R}), 0 \leq \text{loss input}$
9. assumption: ConvexOn $\mathbb{R}$ Set.univ loss
10. assumption: negativeInput < 0
11. assumption: loss negativeInput < loss 0
12. assumption: GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
(GeEtAl2024AlignmentAxioms.coreLoss (GeEtAl2024AlignmentAxioms.weightedLoss (↑weight) loss)) base
13. assumption: $\forall (\text{parameter} : \mathbb{R} \times \mathbb{R}),$
GeEtAl2024AlignmentAxioms.IsGlobalMinimizer
(GeEtAl2024AlignmentAxioms.coreLoss (GeEtAl2024AlignmentAxioms.weightedLoss (↑weight) loss)) parameter →
GeEtAl2024AlignmentAxioms.copyBelowOriginalCone δ parameter
14. conclusion: $\exists (\text{bound} : \mathbb{R}) (\text{radius} : \mathbb{R}),$
0 < bound ∧
0 < radius ∧
 $\forall (\varepsilon : \mathbb{R}),$
0 < ε →
ε < radius →
sInf
(GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective ε δ (↑weight) loss "
{parameter : $\mathbb{R} \times \mathbb{R}$ |
AppliedModelingLib.Alignment.Axioms.linearReward
(GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
(GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures ε δ) 5 ≤
AppliedModelingLib.Alignment.Axioms.linearReward
(GeEtAl2024AlignmentAxioms.twoCoordinateParameter parameter)
(GeEtAl2024AlignmentAxioms.sixCandidateCopyFeatures ε δ) 4}) >
sInf (Set.range (GeEtAl2024AlignmentAxioms.sixCandidateSourceObjective ε δ (↑weight) loss))

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.lemma3_5_correctedPerturbationGap

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The selected source route supplies the archival Lemma 3.5, complete approved corrected statement, defect GE24-LEMMA35-WEAK-HALFSPACE-02, scope note, and approval excerpt/reference. The target takes the approved strict core-minimizer cone and concludes a strict infimum gap for every sufficiently small positive epsilon over the complete weak bad half-space, versus the unrestricted literal six-candidate objective. Its displayed feature family, weighted two-ranking objective, and weak reward comparison are the source construction, so the corrected target is fully bound to the approved scope.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 6

Source locator: cited publication, lines 389-390

Verbatim source input

Theorem 3.6. Fix a nondecreasing loss function ell with $\text{ell}(0) > \inf x \text{ ell}(x)$. If a linear rank aggregation rule f minimizes ell in the majority formulation, then f satisfies PMC.

[Next verbatim source excerpt]

3.2 Majority-Based Loss Formulation

Despite the negative results for loss-function-based rules, we may hope for a remedy using slightly different information. Specifically, we consider a similar loss-based function that rather than getting penalized for disagreeing with each voter only gets penalized if it disagrees with a majority of voters. That is, we choose θ minimizing

$$\text{Lmaj}(\theta; \pi, \text{ell}) = \sum_{a=b \in C} \mathbb{I}[w_{a>b}(\pi) > 1/2] \cdot \text{ell}(r_\theta(b) - r_\theta(a)).$$

Defining a parameter aggregation function based on this loss suffers from the same caveat as before, that in some cases no optimal θ exists. Nevertheless, we can apply an analogous fix for a ranking variant. We say that a linear rank aggregation rule f minimizes ell in the majority formulation if for all $\sigma = f(\pi)$,

$$\inf_{\theta: \theta \in R} \sigma \text{Lmaj}(\theta; \pi, \text{ell}) = \inf_{\theta} \text{Lmaj}(\theta; \pi, \text{ell}).$$

We first show (in Appendix A.5) that this does indeed help achieve PMC with essentially all loss functions.

Theorem 3.6. Fix a nondecreasing loss function ell with $\text{ell}(0) > \inf x \text{ ell}(x)$. If a linear rank aggregation rule f minimizes ell in the majority formulation, then f satisfies PMC.

[Next verbatim source excerpt]

A.5 Proof of Theorem 3.6

Proof. Without loss of generality, we may assume that $\inf x \text{ ell}(x) = 0$, as otherwise we could translate ell without affecting the optimization problem. Fix a profile π with feasible PMC ranking σ , and let $\theta \text{ P M C}$ be a non-degenerate parameter vector that induces σ .

First, we show that $\inf_{\theta} \text{Lmaj}(\theta; \pi, \text{ell}) = 0$. Indeed, note that $c \cdot \theta \text{ P M C} \in R_{\pi}$ for all $c > 0$. Further, note that for any a, b with $w_{a>b}(\pi) > 1/2$, $r_{\theta \text{ P M C}}(b) - r_{\theta \text{ P M C}}(a) < 0$. Therefore, by making c large, the nonzero terms in Lmaj will have an input to ell negative and becoming arbitrarily large in magnitude. Since ell is nondecreasing, these approach the infimum of 0.

Next, for any $\sigma' \neq \sigma$, $\inf_{\theta} \text{Lmaj}(\theta; \pi, \text{ell}) \geq \text{ell}(0)$. Indeed, there must be some pair of candidates a, b

with $a >_{\sigma} b$ and $b >_{\sigma'} a$. For any $\theta \in R_{\sigma}$, $r_{\theta}(b) \geq r_{\theta}(a)$, so $\text{ell}(r_{\theta}(b) - r_{\theta}(a)) \geq \text{ell}(0)$, and this lower bounds the loss function.

Expanded PaperInterface specification

```
∀ {Voter : Type} {inst : Fintype Voter} {n dimension : ℕ}
(features :
  AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension)
(loss : ℝ → ℝ)
(rule :
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features)),
(∀ (input : ℝ), 0 ≤ loss input) →
  Monotone loss →
  loss 0 > sInf (Set.range loss) →
  GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing features loss rule →
  AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) rule
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.FeatureVector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing](#)

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Voter
3. parameter: ℕ
4. parameter: ℕ
5. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.Alignment.Axioms.FeatureVector dimension
6. parameter: ℝ → ℝ
7. parameter: AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features)

8. assumption: $\forall$ (input : $\mathbb{R}$), $0 \leq$ loss input
9. assumption: Monotone loss
10. assumption: loss $0 > \text{slnf}$ (Set.range loss)
11. assumption: GeEtAl2024AlignmentAxioms.IsMajorityLossMinimizing features loss rule
12. conclusion: AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking features) rule

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theorem3_6_majorityLoss

Recorded source-to-Spec screening Verdict: matches

Reason: The source’s nonnegative nondecreasing loss premise and strict loss(0) > inf loss inequality are retained exactly. The target’s IsMajorityLossMinimizing expands to equality of the output-ranking restricted infimum and global infimum for every feasible profile, while majorityLoss sums exactly the source indicator for each strict pairwise majority. The conclusion is PairwiseMajorityConsistent with the source feasibility condition on both profile and PMC ranking.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 459

Verbatim source input

Theorem 4.3. LCPO satisfies PO, PMC, majority consistency and winner monotonicity.

[Next verbatim source excerpt]

4 Social Choice-Based Rule

In light of the above negative results, in this section, we ask whether there are linear rank aggregation rules that concurrently satisfy our two core axioms, PO and PMC. We answer this question affirmatively by presenting a new method based on a prominent rule from voting theory. The Copeland rule assigns a Copeland score to each alternative equal to the number of other alternatives it beats in a pairwise competition, i.e., the score for a is $|\{b \mid w_{a,b} > 1/2\}|$. It then ranks the candidates in descending order according to their Copeland scores (breaking ties arbitrarily). It

7

[form-feed in source extraction]

is known that Copeland satisfies PO, PMC, and additional axiomatic properties. However, in linear social choice, since not every ranking is feasible, we cannot always output the Copeland ranking. We, therefore, define a new linear rank aggregation rule, which we call leximax Copeland. This rule chooses a feasible ranking as follows. It ranks first the candidate with the highest Copeland score that can be feasibly ranked first under some parameter vector θ . Subject to this first position, it ranks second the candidate with the highest Copeland score which can be feasibly ranked second, and continues this process for subsequent positions. Copeland's rule is a C1 rule because it only requires the majority relationships between the candidates. Analogously, leximax Copeland is also a C1 linear rank aggregation rule. Therefore, by Theorem 3.7, it does not satisfy the PO criterion. To address this issue, we define a variant called leximax Copeland subject to PO (LCPO), which incorporates the PO criterion. Under LCPO, for every pair of alternatives where one dominates the other, the rule restricts rankings to place the dominating alternative above the dominated one. The rule remains well-defined since the set of feasible rankings when enforcing the PO criterion is non-empty, as whenever a dominates b , all the rankings in the input profile rank a above b . Note that if the Copeland ranking is feasible, then this rule outputs that ranking, since unrestricted Copeland satisfies PO. In addition to PO and PMC, we wish to show that LCPO satisfies two additional properties, which we define presently. Definition 4.1 (majority consistency). A linear rank aggregation rule satisfies majority consistency if when a candidate a is ranked first by a majority of voters in the input profile, a is ranked first in the output ranking.

Majority consistency ensures that the collective decision reflects the preference of the majority when there is a clear favorite. This principle aligns with PMC, but specifically focuses on the majority's favorite alternative. However, as we discussed above, a PMC ranking does not necessarily exist, and even when it exists, it is not necessarily feasible. By contrast, when a majority winner exists, this candidate is necessarily ranked first by a majority of voters in the input profile, who themselves (by assumption) submit feasible rankings. Therefore, we need not handle the case where it is impossible to rank the majority winner first. Definition 4.2 (winner monotonicity). A linear rank aggregation rule satisfies winner monotonicity if, when a candidate a is ranked first in the output ranking, elevating a in any voter's preference does not cause a to lose their top position in the updated aggregate ranking.

Winner monotonicity ensures that improving a leading candidate's position among individual voters will not result in that candidate's demotion. We now state and prove the main result of this section. Theorem 4.3. LCPO satisfies PO, PMC, majority consistency and winner monotonicity.

[Next verbatim source excerpt]

Proof. LCPO trivially satisfies PO since it always outputs a ranking that respects the PO criterion. Moreover, since Copeland satisfies PMC, and whenever Copeland's ranking is in the domain, leximax Copeland subject to PO returns this ranking, it clearly satisfies PMC. Note that if an alternative a is ranked first by at least half of the voters, then a has the highest Copeland score, meaning that leximax Copeland subject to PO will rank this candidate first if this is possible. We see that this is indeed possible, by noticing that there is at least one feasible ranking in the input profile where a is ranked first, and any such input ranking is feasible (by assumption) and satisfies the PO requirement. Therefore, majority consistency is satisfied. It remains to show that LCPO satisfies winner monotonicity. Suppose that on input profile π , the rule outputs a ranking σ where candidate a is ranked first. Now, consider a profile π' which is similar to π with the only exception being a ranking in which a is placed in a higher position. Let S be the set of agents that ranked above a in Copeland's ranking under π and let S' be the set of agents that ranked above a in Copeland's ranking under π' . Note that $S' \subseteq S$, since when moving from π to π' , only the Copeland score of a can increase, and therefore it is not possible for a candidate b to beat a under π' but not under π .

8

[form-feed in source extraction]

Now, suppose that R and R' are the set of rankings that satisfy the PO criterion with respect to π and π' , respectively. We show that $R' \subseteq R$. First, note that since a is ranked first under π , no alternative dominates a in π , as otherwise the PO criterion would be violated. Therefore, we get that no other alternative dominates a in π' as well. Moreover, note that if b dominates c in π , then this remains true in π' as well. On the other hand, it is possible that a dominates an alternative b in π' but not in π . From all of the above, we conclude that $R' \subseteq R$. Since a is ranked first in σ , we get that for every candidate $b \in S$, there is no ranking in R in which b is ranked first, since otherwise, LCPO would output such a ranking. This also means that for every candidate b in S' , there is no ranking in R' in which b is ranked first, since $R' \subseteq R$ and $S' \subseteq S$. Moreover, note that every ranking in R in which a is ranked first is also in R' since it satisfies all the PO restrictions of π' . Therefore, under π' , LCPO outputs a ranking in which a is ranked first.

Leximax Copeland subject to PO can be implemented in polynomial time by solving $O(|C|^2)$ relatively small linear programs. Specifically, given an input profile, we sequentially choose the candi-

date that is ranked in position $r + 1$ as follows. We denote by σ the partial ranking, where the first r positions have been fixed. For each candidate c that has not been ranked yet, we want to check if there is a parameter vector that adheres to the partial ranking σ , respects the Pareto optimality criterion and ranks c at position $r + 1$. Since all these constraints can be expressed as pairwise comparisons, we can use a linear program such as the one described in Footnote 4 to check if such a feasible ranking exists. Among the candidates meeting this criterion, we select the one with the highest Copeland score for position r .

Settled source reading The checked Theorem 4.3 LCPO rule uses one fixed profile-independent injective candidate priority for equal Copeland scores. This is a declared source clarification, not an inference from single-valuedness.

Expanded PaperInterface specification

```

∀ {Voter : Type} [inst : Fintype Voter] [Nonempty Voter] {n : ℕ}
(feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ),
GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOSelector feasible rule priority →
  AppliedModelingLib.Alignment.Axioms.ParetoOptimal feasible rule ∧
  AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible rule ∧
  AppliedModelingLib.Alignment.Axioms.MajorityConsistent feasible rule ∧
  AppliedModelingLib.Alignment.Axioms.WinnerMonotonic feasible rule

```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.MajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.ParetoOptimal](#)
- [AppliedModelingLib.Alignment.Axioms.WinnerMonotonic](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOSelector](#)

Lean-elaborated claim roles

1. parameter: Type
2. parameter: Fintype Voter
3. assumption: Nonempty Voter
4. parameter: ℕ
5. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop
6. parameter: AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ
8. assumption: GeEtAl2024AlignmentAxioms.IsPriorityTieLCPOSelector feasible rule priority
9. conclusion: AppliedModelingLib.Alignment.Axioms.ParetoOptimal feasible rule ∧
AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible rule ∧
AppliedModelingLib.Alignment.Axioms.MajorityConsistent feasible rule ∧
AppliedModelingLib.Alignment.Axioms.WinnerMonotonic feasible rule

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theorem4_3_constructedFixedTieLCPO

Recorded source-to-Spec screening Verdict: matches

Reason: The conclusion is exactly the source conjunction PO, PMC, majority consistency, and winner monotonicity. IsPriorityTieLCPOSelector expands the source sequential LCPO choice over Pareto-feasible prefix extensions and maximal Copeland score. The displayed approved context confines its injective, profile-independent priority to score ties for these declarations; strict score comparisons and the Pareto feasibility restriction are unchanged, so this is the approved clarification rather than a corrected target.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix results

Review row 8

Source locator: cited publication, lines 1398-1399

Verbatim source input

Theorem C.2. Both Copeland (in traditional social choice) and LCPO (in linear social choice) fail separability.

[Next verbatim source excerpt]

Proof. Consider the following two profiles on 7 candidates with five and three voters each:

Rank	v1	v2	v3	v4	v5	Rank	v6	v7	v8
1	a	b	b	c	d	1	a	b	c
2	g	a	a	e	c	2	d	a	a
3	d	c	e	f	f	3	b	e	e
4	e	e	d	g	g	4	f	d	b
5	f	d	f	b	b	5	c	c	f
6	b	g	g	a	e	6	g	g	g
7	c	f	c	d	a	7	e	f	d

In the first profile, the Copeland scores are 5, 4, 3, 3, 3, 2, 1 for candidates a, b, c, d, e, f, g, respectively. Similarly, in the second profile, they are 6, 5, 3, 3, 3, 1, 0. So under any consistent tie-breaking rule, both of these profiles would output the ranking $a > b > c > d > e > f > g$. However, if we combine these two profiles, then the score of a is 5 while the score of b is 6, and thus, b will be ranked above a, violating separability. To see that this also holds for LCPO, note that when every ranking is feasible, LCPO coincides with Copeland. We can simply have 7 candidates in R7 all located at unit vectors, and voters with inputs from the above profiles.

Expanded PaperInterface specification

```
(∀
(rule :
(voterCount : ℕ) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible),
GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector rule →
¬AppliedModelingLib.Alignment.Axioms.RankingSeparability GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible rule) ∧
∀
(copelandRule lcpoRule :
(voterCount : ℕ) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible),
GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenLCPOSelector copelandRule lcpoRule →
¬AppliedModelingLib.Alignment.Axioms.RankingSeparability GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible lcpoRule
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.RankingSeparability](#)
- [GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector](#)
- [GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenLCPOSelector](#)
- [GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible](#)

Lean-elaborated claim roles

```
1. conclusion: (∀
(rule :
(voterCount : ℕ) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible),
GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenCopelandSelector rule →
¬AppliedModelingLib.Alignment.Axioms.RankingSeparability GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible rule) ∧
∀
(copelandRule lcpoRule :
(voterCount : ℕ) →
AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 5
GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible),
GeEtAl2024AlignmentAxioms.IsConsistentlyTieBrokenLCPOSelector copelandRule lcpoRule →
¬AppliedModelingLib.Alignment.Axioms.RankingSeparability GeEtAl2024AlignmentAxioms.c2AllRankingsFeasible lcpoRule
```

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theoremC_2_copelandAndLCPOFailSeparability

Recorded source-to-Spec screening Verdict: matches

Reason: The target uses Candidate 5, hence the seven candidates of the source example, and its all-rankings-feasible domain is the source unit-vector linear reduction. It states failure of literal profile-append separability for every consistently tie-broken Copeland family and for the corresponding LCPO family. The supporting selectors encode the source's strict Copeland-score ordering and its stated consistent tie-breaking/coincidence condition, with no stronger score comparison substituted.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 1450

Verbatim source input

Theorem C.3. Linear Kemeny satisfies PMC and separability.

[Next verbatim source excerpt]

[form-feed in source extraction]

C.2 Linear Kemeny Rule

Next, we consider a different rule from social choice theory, the Kemeny rule, which can be transformed to the linear setting while maintaining the separability can be achieved along with PMC. Given an input profile π , the Kemeny rule returns a ranking σ^* that minimizes the total number of pairwise disagreements with voters rankings, i.e.

$$\sigma^* \in \arg \min_{\sigma \in S^m} \sum_{i \in V(a,b): a > \sigma_i b} 1(b > \sigma a)$$

where S contains all possible permutations of the m candidates. This expression can be equivalently written as

$$\sigma^* \in \arg \min_{\sigma \in S^m} \sum_{a > b \in C} n_{a > b}(\pi) \cdot 1(b > \sigma a).$$

Here, we define the linear Kemeny rule, which outputs a parameter vector θ^* that induces a ranking that minimizes the total number of pairwise disagreements with voters' rankings, i.e.,

$$\theta^* \in \arg \min_{\theta \in \mathbb{R}^d} \sum_{a > b \in C} n_{a > b}(\pi) \cdot 1(r_\theta(b) > r_\theta(a)).$$

Note that this rule conforms to the standard loss formulation, where the loss function is binary: it is 0 if two rankings agree with respect to the relative ranking of a pair of candidates and 1 otherwise. Since binary loss is not convex, it does not fit in the impossibility result of Theorem 3.1.

Note that Kemeny is generally NP-hard to compute; however, even for linear Kemeny, there is at least an exponential time algorithm by brute-force computing the score of every ranking, and determining whether or not it is feasible.

Theorem C.3. Linear Kemeny satisfies PMC and separability.

Proof. PMC holds because the linear Kemeny score minimizes disagreements even among non-transitive rankings, making the PMC ranking the optimal choice whenever it is feasible. Separability is evident as the Kemeny score of a ranking over two datasets is simply the sum of the scores in each dataset. If the same ranking minimizes the score in both datasets independently, it will also minimize the score in their combination.

Settled source reading The checked Theorem C.3 linear-Kemeny rule is contraction-consistent: a selected tied minimizer remains selected if it stays available after other tied minimizers are removed. This is the source-definedness condition used by the additive separability argument; fixed priority is a checked special case.

Expanded PaperInterface specification

```
∀ {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : (voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible),
AppliedModelingLib.Alignment.Axioms.IsContractionConsistentKemenySelector feasible rule →
(∀ (voterCount : ℕ), AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible (rule voterCount)) ∧
AppliedModelingLib.Alignment.Axioms.RankingSeparability feasible rule
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.IsContractionConsistentKemenySelector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.RankingSeparability](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{N}$
2. parameter: $\text{AppliedModelingLib.SocialChoice.Ranking.Ranking } n \rightarrow \text{Prop}$
3. parameter: $(\text{voterCount} : \mathbb{N}) \rightarrow \text{AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule } (\text{Fin voterCount}) \text{ } n \text{ feasible}$
4. assumption: $\text{AppliedModelingLib.Alignment.Axioms.IsContractionConsistentKemenySelector feasible rule}$
5. conclusion: $(\forall (\text{voterCount} : \mathbb{N}), \text{AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible (rule voterCount)}) \wedge \text{AppliedModelingLib.Alignment.Axioms.RankingSeparability feasible rule}$

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theoremC_3_linearKemeny

Recorded source-to-Spec screening Verdict: matches

Reason: Selected source item 'theoremC_3' together with approved review context GE24-C3-CONTRACTION-TIE-CONVENTION-01. The source defines linear Kemeny by minimizing total pairwise disagreements over feasible rankings and concludes PMC and separability. The displayed Lean prerequisites define the same voter-summed reversal count and feasible argmin, make PMC return a feasible ranking that realizes every strict pairwise majority on a feasible profile, and state separability as equality of the combined-profile output with the common output on the two component profiles. The theorem quantifies over every finite candidate size, feasible-ranking domain, and voter-count-indexed deterministic rule, so this generality includes every source instance. Its hypothesis requires the rule always to select a Kemeny minimizer and applies contraction consistency only when deleting other tied minimizers while the previously selected minimizer remains available; this is exactly the approved source-definedness tie convention and changes no strict Kemeny comparison. The conclusion therefore matches both source conclusions under that recorded source-equivalent clarification.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 1457

Verbatim source input

Theorem C.4. Linear Kemeny does not satisfy PO or majority consistency.

[Next verbatim source excerpt]

Proof. Consider the scenario with 20 candidates whose feature vectors are represented in the table below:

Candidate	Feature Vector
1	(2000000, 0, 0, 0, 0, 0, 0)
2	(0, 2000000, 0, 0, 0, 0, 0)
3	(0, 200000, 0, 0, 0, 0, 0)
4	(0, 100000, 100000, 0, 0, 0, 0)
5	(0, 0, 200000, 0, 0, 0, 0)
6	(0, 0, 20000, 0, 0, 0, 0)
7	(0, 0, 10000, 10000, 0, 0, 0)
8	(0, 0, 0, 20000, 0, 0, 0)
9	(0, 0, 0, 2000, 0, 0, 0)
10	(0, 0, 0, 1000, 1000, 0, 0)
11	(0, 0, 0, 0, 2000, 0, 0)
12	(0, 0, 0, 0, 200, 0, 0)
13	(0, 0, 0, 0, 100, 100, 0)
14	(0, 0, 0, 0, 0, 200, 0)
15	(0, 0, 0, 0, 0, 20, 0)
16	(0, 0, 0, 0, 0, 10, 10)
17	(0, 0, 0, 0, 0, 0, 20)
18	(0, 0, 0, 0, 0, 0, 2)
19	(1, 0, 0, 0, 0, 0, 1)
20	(2, 0, 0, 0, 0, 0, 0)

25

[form-feed in source extraction]

Each vector is constructed such that candidates are prioritized based on the magnitude of their first non-zero entry, leading to a natural ordering within grouped subsets: $\{1, 2\} > \{3, 4, 5\} > \{6, 7, 8\} > \{9, 10, 11\} > \{12, 13, 14\} > \{15, 16, 17\} > \{18, 19, 20\}$. We will have six voters, with rankings induced by the parameter vectors described below.

Voter	Parameter Vector
v1	(2, 1, 7, 6, 5, 4, 3)
v2	(3, 2, 1, 7, 6, 5, 4)
v3	(4, 3, 2, 1, 7, 6, 5)
v4	(5, 4, 3, 2, 1, 7, 6)
v5	(6, 5, 4, 3, 2, 1, 7)
v6	(7, 6, 5, 4, 3, 2, 1)

Each voter ranks candidates within the group in increasing order except for a single reversed group. For instance, v1 ranks candidates as $1 > 2 > 5 > 4 > 3$ and so forth. Under this setup, no parameter vector θ can create a ranking that satisfies all voters' preferences due to the cyclic nature and individual group preferences.

We can check that linear Kemeny rule outputs a ranking in which candidate 2 is ranked above 1. From this analysis, we also conclude that the rules does not satisfy majority consistency either.

Expanded PaperInterface specification

```

V
(rule :
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin 6) 18
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)),
AppliedModelingLib.Alignment.Axioms.IsKemenySelector
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule →
  →AppliedModelingLib.Alignment.Axioms.ParetoOptimal
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule
  →AppliedModelingLib.Alignment.Axioms.MajorityConsistent
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule

```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.IsKemenySelector](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.MajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.ParetoOptimal](#)
- [GeEtAl2024AlignmentAxioms.c4Features](#)

Lean-elaborated claim roles

1. parameter: AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin 6) 18
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)
2. assumption: AppliedModelingLib.Alignment.Axioms.IsKemenySelector
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule
3. conclusion: $\neg$ AppliedModelingLib.Alignment.Axioms.ParetoOptimal
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule $\wedge$
 $\neg$ AppliedModelingLib.Alignment.Axioms.MajorityConsistent
(AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theoremC_4_linearKemenyFailsParetoAndMajorityConsistency

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded target quantifies a Kemeny selector on the displayed six-voter, twenty-candidate construction and concludes failure of both PO and majority consistency. Candidate 18 expands to twenty candidates; the supporting c4 feature matrix is the source's full seven-dimensional table and the six rankings are its six single-group reversals. IsKemenySelector uses the source pairwise-disagreement objective over feasible rankings, so neither axiom nor model domain changes.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 1509

Verbatim source input

Theorem C.5. Linear Kemeny subject to PO violates separability and majority consistency.

[Next verbatim source excerpt]

A possibly easy fix to the problem that linear Kemeny does not satisfy PO would be to enforce the PO criterion, as we did for the LCPO, i.e., to restrict to parameter vectors that respect PO. However, we show that if we do that, then linear Kemeny subject to PO does not satisfy separability anymore. Theorem C.5. Linear Kemeny subject to PO violates separability and majority consistency.

Proof. First, consider a set of candidates and a profile similar to the one that is given in the proof of Theorem C.4. When we restrict to reward functions that 1 above 2, then we can check that Linear Kemeny subject to PO outputs one of the rankings that are in the input profile. Without loss of generality, assume that it outputs the ranking of v_1 . Second, consider the same set of candidates and three voters, with rankings induced by the following parameter vectors:

Voter	Parameter Vector
v_1'	(2, 1, 7, 6, 5, 4, 3)
v_2'	(2, 1, 7, 6, 5, 4, 3)
v_3'	(1, 7, 6, 5, 4, 3, 2)

In this case, Linear Kemeny subject to PO outputs the ranking of v_1' which is the same with this of voter v_1 .

When the two profiles are combined, then we do not anymore restrict on rankings in which 1 is above 2, since 1 does not Pareto dominates 2 anymore. Then, we can check that linear Kemeny subject to PO outputs a different ranking than before in which 2 is ranked above 1. From this example, we see that linear Kemeny subject to PO still violates majority consistency.

[Next verbatim source excerpt]

Proof. Consider the scenario with 20 candidates whose feature vectors are represented in the table below:

Candidate	Feature Vector
1	(2000000, 0, 0, 0, 0, 0, 0)
2	(0, 2000000, 0, 0, 0, 0, 0)
3	(0, 200000, 0, 0, 0, 0, 0)
4	(0, 100000, 100000, 0, 0, 0, 0)
5	(0, 0, 200000, 0, 0, 0, 0)
6	(0, 0, 20000, 0, 0, 0, 0)
7	(0, 0, 10000, 10000, 0, 0, 0)
8	(0, 0, 0, 20000, 0, 0, 0)
9	(0, 0, 0, 2000, 0, 0, 0)
10	(0, 0, 0, 1000, 1000, 0, 0)
11	(0, 0, 0, 0, 2000, 0, 0)
12	(0, 0, 0, 0, 200, 0, 0)
13	(0, 0, 0, 0, 100, 100, 0)
14	(0, 0, 0, 0, 0, 200, 0)
15	(0, 0, 0, 0, 0, 20, 0)
16	(0, 0, 0, 0, 0, 10, 10)
17	(0, 0, 0, 0, 0, 0, 20)
18	(0, 0, 0, 0, 0, 0, 2)
19	(1, 0, 0, 0, 0, 0, 1)
20	(2, 0, 0, 0, 0, 0, 0)

25

[form-feed in source extraction]

Each vector is constructed such that candidates are prioritized based on the magnitude of their first non-zero entry, leading to a natural ordering within grouped subsets: $\{1, 2\} > \{3, 4, 5\} > \{6, 7, 8\} > \{9, 10, 11\} > \{12, 13, 14\} > \{15, 16, 17\} > \{18, 19, 20\}$. We will have six voters, with rankings induced by the parameter vectors described below.

Voter	Parameter Vector
v_1	(2, 1, 7, 6, 5, 4, 3)
v_2	(3, 2, 1, 7, 6, 5, 4)
v_3	(4, 3, 2, 1, 7, 6, 5)
v_4	(5, 4, 3, 2, 1, 7, 6)
v_5	(6, 5, 4, 3, 2, 1, 7)
v_6	(7, 6, 5, 4, 3, 2, 1)

Each voter ranks candidates within the group in increasing order except for a single reversed group. For instance, v_1 ranks candidates as $1 > 2 > 5 > 4 > 3$ and so forth. Under this setup, no parameter vector θ can create a ranking that satisfies all voters' preferences due to the cyclic nature and individual group preferences.

We can check that linear Kemeny rule outputs a ranking in which candidate 2 is ranked above 1. From this analysis, we also conclude that the rules does not satisfy majority consistency either.

Settled source reading The checked C.5 proof derives the source proof's finite first-output classification from the Pareto-constrained Kemeny equality case: the output is one of six displayed ballots. Each possible output has a matching appendage, so the source theorem has no added selector premise or global tie key.

Expanded PaperInterface specification

$\forall$
(rule :

```

(voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 18
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)),
GeEtAl2024AlignmentAxioms.IsParetoKemenySelector rule →
  →AppliedModelingLib.Alignment.Axioms.RankingSeparability
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule ∧
  →AppliedModelingLib.Alignment.Axioms.MajorityConsistent
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) (rule 9)

```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.MajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.RankingSeparability](#)
- [GeEtAl2024AlignmentAxioms.IsParetoKemenySelector](#)
- [GeEtAl2024AlignmentAxioms.c4Features](#)

Lean-elaborated claim roles

```

1. parameter: (voterCount : ℕ) →
  AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) 18
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features)
2. assumption: GeEtAl2024AlignmentAxioms.IsParetoKemenySelector rule
3. conclusion: →AppliedModelingLib.Alignment.Axioms.RankingSeparability
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) rule ∧
  →AppliedModelingLib.Alignment.Axioms.MajorityConsistent
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c4Features) (rule 9)

```

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theoremC_5_paretoKemenyFailsSeparabilityAndMajorityConsistency

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem C.5 (source transcript 1506-1528) asserts that linear Kemeny restricted to PO violates separability and majority consistency. The expanded Spec quantifies the finite-profile rule family on the 20-candidate C.4 feature instance, assumes exactly 'IsParetoKemenySelector rule' (linear-feasible, Pareto-respecting Kemeny minimization on every source-feasible profile), and concludes failure of 'RankingSeparability' and of 'MajorityConsistent' at the nine-voter component. It has no 'HasC5SourceBallotOutput', chosen-v1, or fixed-tie premise. In the current C.5 route, 'c4ParetoKemenySelector hasC5SourceBallotOutput' derives the six-ballot first-profile classification from that selector premise, and the terminal theorem supplies this derivation internally; the queue's approved source-proof-realization context therefore does not change the source claim or add an assumption.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 1543-1544

Verbatim source input

Theorem C.6. Leximax Plurality satisfies majority consistency, winner monotonicity and separability.

[Next verbatim source excerpt]

C.3 Leximax Plurality

Plurality is probably the most ubiquitous voting rule in the world. Its ranking variant ranks the candidates in decreasing order with respect to their plurality scores. The plurality score of a candidate is equal to the number of her appearances in the first position. This rule is known to satisfy several axioms but in linear social choice cannot be directly applied, as not all rankings are feasibly. Similarly to leximax Copeland, we define Leximax Plurality as follows. It ranks first the candidate with the highest plurality score that can be ranked first under some parameter vector. Subject to this first position, it ranks second the candidate with the highest plurality score that can feasibly be ranked second, and so on, until all the positions are filled.

26

[form-feed in source extraction]

Theorem C.6. Leximax Plurality satisfies majority consistency, winner monotonicity and separability.

Proof. Note that leximax Plurality always returns a ranking in which the candidate with the highest plurality score is ranked first, since there exists at least one feasible ranking in which this candidate is ranked first. From this observation, we immediately see that leximax Plurality satisfies majority consistency.

Now, suppose that on input profile π , the rule outputs a ranking such that candidate a is ranked first. This means that a has the highest plurality score. Now, consider a profile π' which is similar to π with the only exception being a ranking in which a is ranked in a higher position. It is clear that a continues to have the highest plurality score and therefore leximax Plurality will output a ranking in which a is ranked first. Therefore, winner monotonicity is satisfied.

It remains to prove that the rule satisfies separability. Suppose that under two different profiles π_1 and π_2 , the rule outputs σ , and under the aggregated profile π_3 , it outputs σ' . We will show that $\sigma = \sigma'$. One main observation is that if a candidate a has a higher plurality score than a candidate b under both π_1 and π_2 , then a has a higher plurality score than b under π_3 as well. We will show the desired property by induction on the positions of the ranking σ . Start from the first position in which say candidate a is ranked first. From above, we know that a has the highest plurality score under both π_1 and π_2 , which remains true in π_3 , and therefore a is ranked first in σ' . Now, assume that up to position $t - 1$, σ and σ' are similar and denote with a' the candidate that is ranked at position t of σ . We denote by S_1 , S_2 and S_3 the set of candidates that are not ranked among the first t positions in σ and have higher plurality score than a' under π_1 , π_2 , and π_3 respectively. Since, a' is ranked at the t -th position in σ , we get that, subject to the fixed first $t - 1$ position, no candidate in $S_1 \cup S_2$ can be ranked at the t -th position. The theorem follows by noticing that $S_3 \subseteq S_1 \cup S_2$, which follows from the main observation above.

27

Settled source reading The checked C.6 rule uses one fixed profile-independent injective candidate priority for equal plurality scores. Strict plurality comparisons are unchanged; the theorem has this specified rule-family scope.

Expanded PaperInterface specification

```
∀ {n : ℕ} (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
(rule : (voterCount : ℕ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible)
(priority : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℕ),
(∀ (voterCount : ℕ),
  GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralitySelector feasible (rule voterCount) priority) →
(∀ (voterCount : ℕ), AppliedModelingLib.Alignment.Axioms.MajorityConsistent feasible (rule voterCount)) ∧
(∀ (voterCount : ℕ), AppliedModelingLib.Alignment.Axioms.WinnerMonotonic feasible (rule voterCount)) ∧
AppliedModelingLib.Alignment.Axioms.RankingSeparability feasible rule
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.MajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.RankingSeparability](#)
- [AppliedModelingLib.Alignment.Axioms.WinnerMonotonic](#)
- [AppliedModelingLib.SocialChoice.Ranking.Candidate](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.IsPriorityTieLeximaxPluralitySelector](#)

Lean-elaborated claim roles

- 1. parameter: $\mathbb{N}$
- 2. parameter: `AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop`
- 3. parameter: `(voterCount :  $\mathbb{N}$ ) → AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin voterCount) n feasible`
- 4. parameter: `AppliedModelingLib.SocialChoice.Ranking.Candidate n →  $\mathbb{N}$`
- 5. assumption: $\forall$ (voterCount : $\mathbb{N}$), `GeEtAI2024AlignmentAxioms.IsPriorityTieLeximaxPluralitySelector feasible (rule voterCount) priority`
- 6. conclusion: $(\forall$ (voterCount : $\mathbb{N}$), `AppliedModelingLib.Alignment.Axioms.MajorityConsistent feasible (rule voterCount)`) $\wedge$ $(\forall$ (voterCount : $\mathbb{N}$), `AppliedModelingLib.Alignment.Axioms.WinnerMonotonic feasible (rule voterCount)`) $\wedge$ `AppliedModelingLib.Alignment.Axioms.RankingSeparability feasible rule`

Lean proof endpoint (built separately)

`GeEtAI2024AlignmentAxioms.theoremC_6_fixedTieLeximaxPlurality`

Recorded source-to-Spec screening Verdict: matches

Reason: The target gives exactly the three source properties: majority consistency, winner monotonicity, and literal append separability. `IsPriorityTieLeximaxPluralitySelector` expands the source sequential feasible-prefix selection by highest plurality count. The approved context confines its single injective, profile-independent candidate priority to equal plurality scores, leaving every strict plurality comparison and feasibility condition unchanged; it is consequently an ordinary clarified match.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 1333-1385

Verbatim source input

B PMC Infeasibility Example

Consider the case with $d = 3$ and seven candidates: one special candidate a^* located at $(1/4, 1/4, 1/4)$, and six others $c_i^\pm$ located at standard basis vectors e_i . We have three voters with parameter vectors $(1, 2\epsilon, \epsilon)$, $(2\epsilon, 1, \epsilon)$, and $(2\epsilon, \epsilon, 1)$, where $\epsilon < 1/5$ is a small positive number. These voters have the following induced rankings:

23			
[form-feed in source extraction]			
Rank	v1	v2	v3
1	c_1^+	c_2^+	c_3^+
2	a^*	a	a^*
3	c_2^+	c_1^+	c_1^+
4	c_3^+	c_3^+	c_2^+
5	c_3^-	c_3^-	c_2^-
6	c_2^-	c_1^-	c_1^-
7	c_1^-	c_2^-	c_3^-

We argue the ranking $a^* > c_1^+ > c_2^+ > c_3^+ > c_3^- > c_2^- > c_1^-$ is a PMC ranking but no linear reward function can position a at the top of this ranking.

For any reward vector $\theta = (\theta_1, \theta_2, \theta_3) \in \mathbb{R}^3$, the reward for a^* is :

$$ra_{\theta^*} = \frac{1}{4} (\theta_1 + \theta_2 + \theta_3)$$

Given the placement of $c_i^\pm$ at the standard basis vectors, each c_i achieves a reward equivalent to one of the absolute values of the components of θ , thus surpassing ra_{θ^*} since $ra_{\theta^*} < \max_i |\theta_i|$.

[Next verbatim source excerpt]

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by $\mathbb{R}^d$ the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in \mathbb{R}^d$. A parameter vector $\theta \in \mathbb{R}^d$ induces a linear reward function $r_\theta : C \rightarrow \mathbb{R}$ defined by taking the dot product with feature vectors $r_\theta(c) = (\theta, x_c)$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a \geq b} = \{\theta \mid r_\theta(a) \geq r_\theta(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a \geq b}$ and $R_{b \geq a}$ split $\mathbb{R}^d$ into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_\theta(a) = r_\theta(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_\sigma b$ implies $r_\theta(a) \geq r_\theta(b)$. Let $R_\sigma = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_\sigma = \bigcap_{a,b: a \succ_\sigma b} R_{a \geq b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_\sigma\}$ essentially form a partition of $\mathbb{R}^d$, covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_\theta(a) \neq r_\theta(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_σ , and thus induce exactly one ranking. We call σ feasible if R_σ has a nonempty interior, i.e., is induced by some nondegenerate θ .

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $na \succ b(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $wa \succ b(\pi) = na \succ b(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $na \succ b$ and $wa \succ b$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will

only be with respect to how $r\theta^*$ ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

In our study, we examine several axioms borrowed from social choice theory to evaluate the reasonableness (fairness) of our aggregation mechanisms. These axioms include:

Definition 2.1 (Pareto Optimality). A linear rank aggregation rule f satisfies Pareto optimality if, whenever every voter prefers candidate a over candidate b on π , i.e., $w_a > b(\pi) = 1$, then candidate a is ranked higher than candidate b in the output ranking, i.e., $a > f(\pi) b$.

Definition 2.2 (Pairwise Majority Consistency (PMC)). A ranking σ is called a PMC ranking for profile π if for all $a, b \in C$, $a >_\sigma b$ if and only if a majority of voters rank $a >_\pi b$, i.e., $w_a > b > 1/2$.

A linear rank aggregation rule satisfies PMC if, when a PMC ranking σ exists for the input profile π and σ is feasible, then $f(\pi) = \sigma$.

Note that a PMC ranking for each π need not exist, but when one does, it is unique. The words “ σ is feasible” allude to the possibility that no non-degenerate parameter vector θ induces the unique PMC ranking. Indeed, we have such an example; see Appendix B for details.

[Next verbatim source excerpt]

B PMC Infeasibility Example

Consider the case with $d = 3$ and seven candidates: one special candidate a^* located at $(1/4, 1/4, 1/4)$, and six others $c_\pm^i$ located at standard basis vectors e_i . We have three voters with parameter vectors $(1, 2\epsilon, \epsilon)$, $(2\epsilon, 1, \epsilon)$, and $(2\epsilon, \epsilon, 1)$, where $\epsilon < 1/5$ is a small positive number. These voters have the following induced rankings:

23			
[form-feed in source extraction]			
Rank	v1	v2	v3
1	c_+^1	c_+^2	c_+^3
	1	2	3
2	a^*	a^*	
3	a	a	a^*
	c_+^2		
	2	c_+^1	c_+^1
4	c_+^3	c_+^3	c_+^2
	3		
5	c_-^3	c_-^3	c_-^2
	3		
6	c_-^2	c_-^1	c_-^1
	2	1	1
7	c_-^1	c_-^2	c_-^3
	1	2	3

We argue the ranking $a^* > c_+^1 > c_+^2 > c_+^3 > c_2^- > c_1^-$ is a PMC ranking but no linear reward

function can position a^* at the top of this ranking.

For any reward vector $\theta = (\theta_1, \theta_2, \theta_3) \in \mathbb{R}^3$, the reward for a^* is :

$$ra\theta^* = \frac{1}{4} (\theta_1 + \theta_2 + \theta_3)$$

Given the placement of $c_\pm^i$ at the standard basis vectors, each c_i achieves a reward equivalent to one of the absolute values of the components of θ , thus surpassing $ra\theta^*$ since $ra\theta^* < \max_i |\theta_i|$.

Expanded PaperInterface specification

```
AppliedModelingLib.Alignment.Axioms.FeasibleProfile
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAI2024AlignmentAxioms.appendixBFeatures)
  GeEtAI2024AlignmentAxioms.appendixBProfile  $\wedge$ 
AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAI2024AlignmentAxioms.appendixBProfile
  GeEtAI2024AlignmentAxioms.appendixBPMCRanking  $\wedge$ 
( $\forall$  (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking 5),
  AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAI2024AlignmentAxioms.appendixBProfile
  ranking  $\rightarrow$ 
  ranking = GeEtAI2024AlignmentAxioms.appendixBPMCRanking)  $\wedge$ 
 $\rightarrow$ AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAI2024AlignmentAxioms.appendixBFeatures
  GeEtAI2024AlignmentAxioms.appendixBPMCRanking
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.FeasibleProfile](#)

- [AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAI2024AlignmentAxioms.appendixBFeatures](#)
- [GeEtAI2024AlignmentAxioms.appendixBPMCRanking](#)
- [GeEtAI2024AlignmentAxioms.appendixBProfile](#)

Lean-elaborated claim roles

1. conclusion: AppliedModelingLib.Alignment.Axioms.FeasibleProfile
 (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAI2024AlignmentAxioms.appendixBFeatures)
 GeEtAI2024AlignmentAxioms.appendixBProfile $\wedge$
 AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAI2024AlignmentAxioms.appendixBProfile
 GeEtAI2024AlignmentAxioms.appendixBPMCRanking $\wedge$
 ($\forall$ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking 5),
 AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAI2024AlignmentAxioms.appendixBProfile
 ranking $\rightarrow$
 ranking = GeEtAI2024AlignmentAxioms.appendixBPMCRanking) $\wedge$
 $\neg$ AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAI2024AlignmentAxioms.appendixBFeatures
 GeEtAI2024AlignmentAxioms.appendixBPMCRanking

Lean proof endpoint (built separately)

GeEtAI2024AlignmentAxioms.appendixBExplicitInfeasiblePMC

Recorded source-to-Spec screening Verdict: matches

Reason: The Appendix B bundle fixes seven candidates in R^3 , the three displayed ballots, the stated PMC order, and the impossibility of placing a^* at its top. The expanded target uses Candidate 5 (seven candidates), the displayed signed-axis-plus-star feature map, and those three ballots; it conjunctively states feasible inputs, PMC equivalence and uniqueness for the displayed ranking, and absence of a nondegenerate linear inducer. Each material definition expands to those source objects without changing a comparison or domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text results (continued)

Review row 14

Source locator: cited publication, lines 204-211

Verbatim source input

Definition 2.2 (Pairwise Majority Consistency (PMC)). A ranking σ is called a PMC ranking for profile π if for all $a, b \in C$, $a \succ_{\sigma} b$ if and only if a majority of voters rank $a \succ_{\sigma} b$, i.e., $w_{a \succ b} > 1/2$. A linear rank aggregation rule satisfies PMC if, when a PMC ranking σ exists for the input profile π and σ is feasible, then $f(\pi) = \sigma$.

Note that a PMC ranking for each π need not exist, but when one does, it is unique. The words “ σ is feasible” allude to the possibility that no non-degenerate parameter vector θ induces the unique PMC ranking. Indeed, we have such an example; see Appendix B for details.

[Next verbatim source excerpt]

2 The Linear Social Choice Model

Let C be a set of m distinct prompt/responses, referred to as candidates, and let $V = \{1, \dots, n\}$ be a set of n human participants, known as voters. We denote by R_d the d -dimensional real space in which both candidate feature vectors and chosen parameter vectors lie.

Each candidate $c \in C$ is associated with a distinct feature vector $x_c \in R_d$. A parameter vector $\theta \in R_d$ induces a linear reward function $r_{\theta} : C \rightarrow \mathbb{R}$ defined by taking the dot product with feature vectors $r_{\theta}(c) = \langle \theta, x_c \rangle$. We will primarily be interested in how these parameterized functions rank the candidates by reward. Let $R_{a \succ b} = \{\theta \mid r_{\theta}(a) \geq r_{\theta}(b)\}$ be the region where the reward of a is at least as large as that of b . Note that $R_{a \succ b}$ and $R_{b \succ a}$ split R_d into two half spaces, separated by the hyperplane orthogonal to $x_a - x_b$. Parameter vectors θ on the hyperplane have $r_{\theta}(a) = r_{\theta}(b)$, while rankings in the interior of either half-space strictly rank one over the other.

For a ranking σ over the candidates, we say that θ induces σ , denoted $\theta \triangleright \sigma$, if $a \succ_{\sigma} b$ implies $r_{\theta}(a) \geq r_{\theta}(b)$. Let $R_{\sigma} = \{\theta \mid \theta \triangleright \sigma\}$ be the set of vectors θ that induce it. Note that this can be written as the intersection of corresponding half spaces $R_{\sigma} = \bigcap_{a \succ_{\sigma} b} R_{a \succ b}$. Further, the

3

[form-feed in source extraction]

collection of $\{R_{\sigma}\}$ essentially form a partition of R_d , covering the space and intersecting only at their boundaries.

We call a θ non-degenerate if it is fully on one side of each of the separating hyperplanes, i.e., $r_{\theta}(a) \neq r_{\theta}(b)$ for all $a, b \in C$. Non-degenerate parameter vectors lie in the interior of some R_{σ} , and thus induce exactly one ranking. We call σ feasible if R_{σ} has a nonempty interior, i.e., is induced by some nondegenerate θ .³

Each voter $i \in V$ submits a ranking over the candidate σ_i . We assume that the feature space is rich enough that voter preferences can be captured via non-degenerate parameter vectors. In other words, we assume that each σ_i is feasible. We refer to the vector of voter rankings $\pi = (\sigma_i)_{i \in V}$ as a profile. Further, for two candidates a, b , we write $n_{a \succ b}(\pi) := |\{i \in V \mid a \succ_{\sigma_i} b\}|$ for the number of voters that prefer a to b , and $w_{a \succ b}(\pi) = n_{a \succ b}(\pi)/n$ for the proportion of such voters. When the profile π is clear from context, we may shorten these to $n_{a \succ b}$ and $w_{a \succ b}$, respectively.

We define a parameter aggregation rule as a function that takes as input a profile π and outputs a parameter vector θ^* . Our goal is to design parameter aggregation rules such that r_{θ^*} satisfies desirable properties with respect to the voter preferences. However, as the properties we care about will only be with respect to how r_{θ^*} ranks the candidates, it will be more convenient to work with what we call linear rank aggregation rules that take as input a profile π and output a feasible ranking σ . There is a natural way to interpret a parameter aggregation rule as a linear rank aggregation rule, namely, output any feasible ranking induced by θ^* . The exact properties of the parameter aggregation rule could in principle be sensitive to the tie-breaking of non-degenerate outputs, however, all of our results will be robust to such tie-breaking.⁴

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b .

In our study, we examine several axioms borrowed from social choice theory to evaluate the reasonableness (fairness) of our aggregation mechanisms. These axioms include:

Definition 2.1 (Pareto Optimality). A linear rank aggregation rule f satisfies Pareto optimality if, whenever every voter prefers candidate a over candidate b on π , i.e., $w_{a \succ b}(\pi) = 1$, then candidate a is ranked higher than candidate b in the output ranking, i.e., $a \succ_f(\pi) b$.

Definition 2.2 (Pairwise Majority Consistency (PMC)). A ranking σ is called a PMC ranking for profile π if for all $a, b \in C$, $a \succ_{\sigma} b$ if and only if a majority of voters rank $a \succ_{\sigma} b$, i.e., $w_{a \succ b} > 1/2$. A linear rank aggregation rule satisfies PMC if, when a PMC ranking σ exists for the input profile π and σ is feasible, then $f(\pi) = \sigma$.

Note that a PMC ranking for each π need not exist, but when one does, it is unique. The words “ σ is feasible” allude to the possibility that no non-degenerate parameter vector θ induces the unique PMC ranking. Indeed, we have such an example; see Appendix B for details.

Expanded PaperInterface specification

```
(¬∃ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking 7),
  AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAl2024AlignmentAxioms.c1ProfilePlus1 ranking) ∧
∀ (Voter : Type) [inst : Fintype Voter] (n : ℕ) (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
  (first second : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile first →
  AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile second → first = second
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)

- [GeEtAl2024AlignmentAxioms.c1ProfilePlus1](#)

Lean-elaborated claim roles

1. conclusion: $(\neg \exists (\text{ranking} : \text{AppliedModelingLib.SocialChoice.Ranking.Ranking } 7),$
 $\text{AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAl2024AlignmentAxioms.c1ProfilePlus1 ranking}) \wedge$
 $\forall (\text{Voter} : \text{Type}) [\text{inst} : \text{Fintype Voter}] (n : \mathbb{N}) (\text{profile} : \text{AppliedModelingLib.Alignment.Axioms.RankingProfile Voter } n)$
 $(\text{first second} : \text{AppliedModelingLib.SocialChoice.Ranking.Ranking } n),$
 $\text{AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile first} \rightarrow$
 $\text{AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile second} \rightarrow \text{first} = \text{second}$

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.pmcRankingUniquenessAndNonexistence

Recorded source-to-Spec screening Verdict: matches

Reason: Definition 2.2's biconditional between a ranking's strict comparisons and strict voter majorities expands unchanged through IsPairwiseMajorityRanking, StrictMajorityPrefers, and the finite strict-majority count. The target gives the two stated consequences: a concrete profile with no PMC ranking and universal uniqueness whenever two rankings satisfy that biconditional. The displayed candidate/profile construction is a nine-candidate source construction establishing nonexistence; no quantifier or majority threshold is altered.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 397-410

Verbatim source input

However, despite this good news for PMC, we show that this does not help in achieving PO. In fact, our negative result extends to every C1 linear rank aggregation rule. Note that if f minimizing ℓ in the majority formulation breaks ties consistently (i.e., if multiple feasible rankings are optimal, then it consistently chooses the same one), then it is C1. We then have the following result, whose proof is relegated to Appendix A.6.

Theorem 3.7. All C1 linear rank aggregation rules fail PO.

This result is quite unfortunate, because if there were a rule that is both C1 and PO, we would automatically achieve PMC: Whenever there is a feasible PMC ranking, a C1 rule cannot distinguish between this profile and a profile where all voters submit this ranking, hence, under the PO criterion, it must output it. Furthermore, whenever there is a PMC ranking, outputting it is necessarily PO, as for every pair, a majority of voters agree with the PMC ranking. Interestingly, in the proof, we construct a profile which has a PMC ranking, yet it is not feasible, and no matter how a C1 linear rank aggregation rule breaks ties, there is an underlying profile in which this output violates PO.

[Next verbatim source excerpt]

We pay special attention to a prominent family of rules from social choice theory referred to as C1 rules [14], whose outputs depend only on majority relationships, i.e., they only need to know for each pair of candidates (a, b) whether the majority prefers a or b . In our study, we examine several axioms borrowed from social choice theory to evaluate the reasonableness (fairness) of our aggregation mechanisms. These axioms include:

Definition 2.1 (Pareto Optimality). A linear rank aggregation rule f satisfies Pareto optimality if, whenever every voter prefers candidate a over candidate b on π , i.e., $w_a > b(\pi) = 1$, then candidate a is ranked higher than candidate b in the output ranking, i.e., $a > f(\pi) b$.

Definition 2.2 (Pairwise Majority Consistency (PMC)). A ranking σ is called a PMC ranking for profile π if for all $a, b \in C$, $a > \sigma b$ if and only if a majority of voters rank $a > b$, i.e., $w_a > b > 1/2$.

A linear rank aggregation rule satisfies PMC if, when a PMC ranking σ exists for the input profile π and σ is feasible, then $f(\pi) = \sigma$.

Note that a PMC ranking for each π need not exist, but when one does, it is unique. The words “ σ is feasible” allude to the possibility that no non-degenerate parameter vector θ induces the unique PMC ranking. Indeed, we have such an example; see Appendix B for details.

[Next verbatim source excerpt]

A.6 Proof of Theorem 3.7

Proof. We construct an explicit instance and pairwise majority relationships such that no matter what feasible ranking a rule picks, there is an underlying profile where that output was a PO violation.

We will have 9 candidates; 8 will be labeled c_i^+ and c_i^- for $i = 1, 2, 3, 4$, and one labeled c^* . They will have feature vectors in $\mathbb{R}^4$. Each c_i will be located at $x c_i \pm e_i$ where e_i is the i 'th

standard basis vector, i.e., c_1^+ is at $(0, 1, 0, 0)$ and c_4^- is at $(0, 0, 0, -1)$. Finally, c^* will be located at $(1/5, 1/5, 1/5, 1/5)$.

There will be 5 voters. Their pairwise majority graph will be as follows. Candidate c^* will pairwise

beat all others. In addition, each c_i^+ will pairwise beat each c_j^- . Among the c_i candidates, there will be a cycle $c_1^+ > c_2^+ > c_3^+ > c_4^+ > c_1^+$ and between the remaining two pairs $c_1^- > c_3^-$ and $c_2^- > c_4^-$.

The c_i^- candidates will be the exact reverse of this, i.e., a cycle $c_4^- > c_3^- > c_2^- > c_1^- > c_4^-$, along with $c_3^- > c_1^-$ and $c_4^- > c_2^-$. A pictorial representation can be found in Figure 1.

A C1 rule must pick a θ solely based on the pairwise majority graph. We will show that regardless of what θ it outputs, this will lead to a PO violation.

22

[form-feed in source extraction]



[Next verbatim source excerpt]

However, despite this good news for PMC, we show that this does not help in achieving PO. In fact, our negative result extends to every C1 linear rank aggregation rule. Note that if f minimizing ℓ in the majority formulation breaks ties consistently (i.e., if multiple feasible rankings are optimal, then it consistently chooses the same one), then it is C1. We then have the following result, whose proof is relegated to Appendix A.6.

Theorem 3.7. All C1 linear rank aggregation rules fail PO.

This result is quite unfortunate, because if there were a rule that is both C1 and PO, we would automatically achieve PMC: Whenever there is a feasible PMC ranking, a C1 rule cannot distinguish between this profile and a profile where all voters submit this ranking, hence, under the PO criterion, it must output it. Furthermore, whenever there is a PMC ranking, outputting it is necessarily PO,

as for every pair, a majority of voters agree with the PMC ranking. Interestingly, in the proof, we construct a profile which has a PMC ranking, yet it is not feasible, and no matter how a C1 linear rank aggregation rule breaks ties, there is an underlying profile in which this output violates PO.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Every C1 linear rank aggregation rule fails Pareto optimality; C1 plus Pareto optimality implies PMC; and any existing PMC ranking respects every
 ↪ unanimous comparison. Correcting the attached proof-location sentence, the Appendix-A.6 profile has a cyclic majority relation and no PMC ranking;
 ↪ Appendix B supplies the separate infeasible-PMC example.

Archival source anchor: cited publication, lines 404-410

Expanded PaperInterface specification

```
(∀
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin 5) 7
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features)),
  AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features) rule →
  →AppliedModelingLib.Alignment.Axioms.ParetoOptimal
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features) rule) ∧
  (∀ (Voter : Type) [inst : Fintype Voter] (n : ℕ) (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
    (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible),
    AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule feasible rule →
    AppliedModelingLib.Alignment.Axioms.ParetoOptimal feasible rule →
    AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible rule) ∧
  (∀ (Voter : Type) [inst : Fintype Voter] [Nonempty Voter] (n : ℕ)
    (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
    (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
    AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile ranking →
    GeEtAl2024AlignmentAxioms.RespectsPareto profile ranking) ∧
  →∃ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking 7),
    AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAl2024AlignmentAxioms.c1ProfilePlus1 ranking)
```

Retained governing declarations

- [AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking](#)
- [AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule](#)
- [AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent](#)
- [AppliedModelingLib.Alignment.Axioms.ParetoOptimal](#)
- [AppliedModelingLib.Alignment.Axioms.RankingProfile](#)
- [AppliedModelingLib.SocialChoice.Ranking.Ranking](#)
- [GeEtAl2024AlignmentAxioms.RespectsPareto](#)
- [GeEtAl2024AlignmentAxioms.c1Features](#)
- [GeEtAl2024AlignmentAxioms.c1ProfilePlus1](#)

Lean-elaborated claim roles

```
1. conclusion: (∀
  (rule :
    AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule (Fin 5) 7
    (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features)),
  AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features) rule →
  →AppliedModelingLib.Alignment.Axioms.ParetoOptimal
  (AppliedModelingLib.Alignment.Axioms.LinearFeasibleRanking GeEtAl2024AlignmentAxioms.c1Features) rule) ∧
  (∀ (Voter : Type) [inst : Fintype Voter] (n : ℕ) (feasible : AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop)
    (rule : AppliedModelingLib.Alignment.Axioms.LinearRankAggregationRule Voter n feasible),
    AppliedModelingLib.Alignment.Axioms.C1LinearRankAggregationRule feasible rule →
    AppliedModelingLib.Alignment.Axioms.ParetoOptimal feasible rule →
    AppliedModelingLib.Alignment.Axioms.PairwiseMajorityConsistent feasible rule) ∧
  (∀ (Voter : Type) [inst : Fintype Voter] [Nonempty Voter] (n : ℕ)
    (profile : AppliedModelingLib.Alignment.Axioms.RankingProfile Voter n)
    (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
    AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking profile ranking →
    GeEtAl2024AlignmentAxioms.RespectsPareto profile ranking) ∧
  →∃ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking 7),
    AppliedModelingLib.Alignment.Axioms.IsPairwiseMajorityRanking GeEtAl2024AlignmentAxioms.c1ProfilePlus1 ranking)
```

Lean proof endpoint (built separately)

GeEtAl2024AlignmentAxioms.theorem3_7_C1FailsParetoAndConsequences

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The route includes the archival text, complete correction statement, defect GE24-THEOREM37-PROOF-WITNESS-LOCATION-08, scope note, and approval excerpt/reference. The expanded target preserves Theorem 3.7 (every C1 selector on the displayed A.6 construction fails PO) and the two attached consequences C1+PO implies PMC and any PMC ranking respects unanimous comparisons. Its final no-PMC assertion is the approved relocation correction for the cyclic A.6 profile; the separate Appendix B target retains the unique infeasible-PMC example.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--